

Day 10 Practice Assignment

Hands-on JUnit 5: Annotations, Assertions, Parameterized Tests and Advanced Concepts

John Michael Bilbao
Techstart
August 20, 2026

Introduction

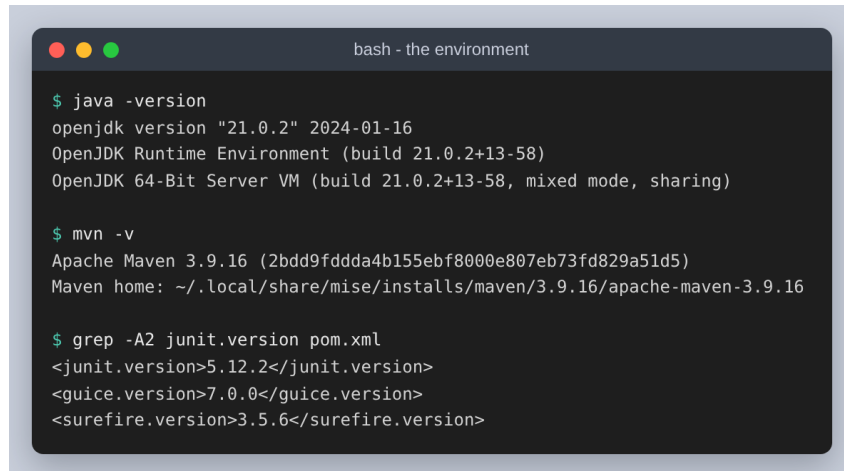
The assignment asks for a calculator with the four arithmetic operations, and a JUnit 5 test suite that exercises the annotations, the assertions, parameterized tests, test suites, dependency injection and the more advanced parts of the framework.

The calculator is small on purpose. Four operations are not interesting to implement, so the effort went into giving them something worth testing: a division that refuses rather than returning infinity, a rounding policy that can be swapped out, and an operation log that records what happened. Those three decisions are what turn the lifecycle annotations, the assertions and the injection from things demonstrated into things needed.

The result is 185 tests across nine test classes. Every figure in this document is output from a real run, produced by the two scripts in the project rather than typed by hand.

What the assignment asked for	Where it is
A Calculator class with the four operations	Calculator.java, Operation.java
@Test with @BeforeEach and @AfterEach	CalculatorTest, LifecycleOrderTest
assertEquals and assertThrows	CalculatorTest, throughout
@ParameterizedTest	CalculatorParameterizedTest, six argument sources
A test suite with @Suite	CalculatorTestSuite
Dependency injection, simple injector or Guice	Both: SimpleInjector.java and CalculatorModule.java
Advanced concepts: nested or dynamic tests	Both: CalculatorNestedTest and CalculatorDynamicTest

The environment. Guice 7 is the first release built on the jakarta.inject annotations, which is why the same annotation works for both Guice and the hand-rolled injector later on.

A terminal window titled "bash - the environment" with a dark background and light-colored text. It shows the output of three commands: 'java -version', 'mvn -v', and 'grep -A2 junit.version pom.xml'.

```
bash - the environment

$ java -version
openjdk version "21.0.2" 2024-01-16
OpenJDK Runtime Environment (build 21.0.2+13-58)
OpenJDK 64-Bit Server VM (build 21.0.2+13-58, mixed mode, sharing)

$ mvn -v
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: ~/.local/share/mise/installs/maven/3.9.16/apache-maven-3.9.16

$ grep -A2 junit.version pom.xml
<junit.version>5.12.2</junit.version>
<guice.version>7.0.0</guice.version>
<surefire.version>3.5.6</surefire.version>
```

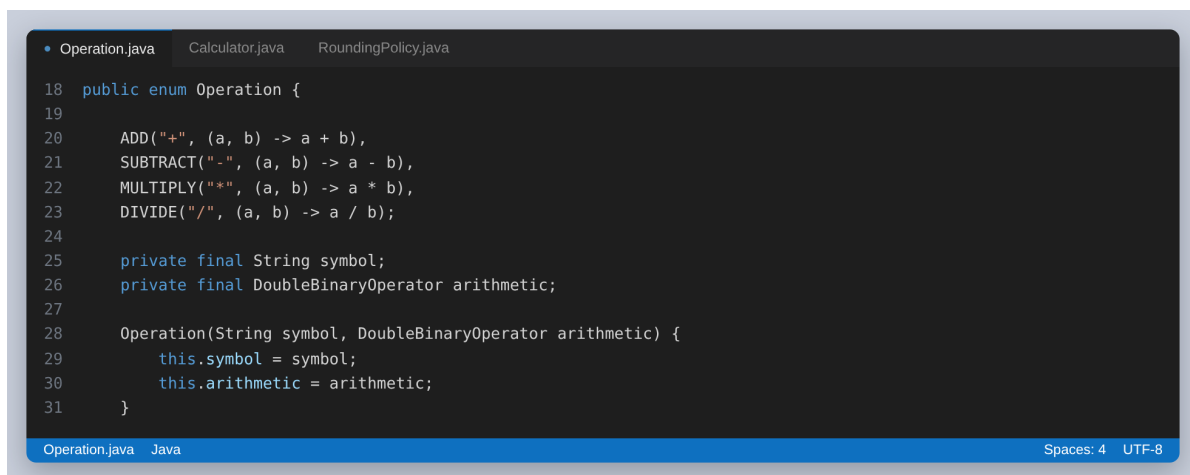
Figure 1: Java, Maven and the three versions that matter

Step 1: The calculator application

Four operations, and a fifth method that all four delegate to. Putting the validation in one place means it cannot be bypassed by calling a different method, and it means there is one thing to read when asking what the calculator refuses.

Method	Result	Refuses
add, subtract, multiply	the arithmetic, rounded	a NaN or infinite operand; a result that overflows
divide	the quotient, rounded	the same, and a zero divisor
apply(Operation, a, b)	any of the four by enum	the same, and a null operation
history()	every operation recorded so far	—

The operations live in an enum, paired with their symbol and the raw calculation. The enum holds no validation at all, deliberately: it is the unguarded arithmetic, and having it available in that form is what lets a test show the difference between what the hardware does and what the calculator does.

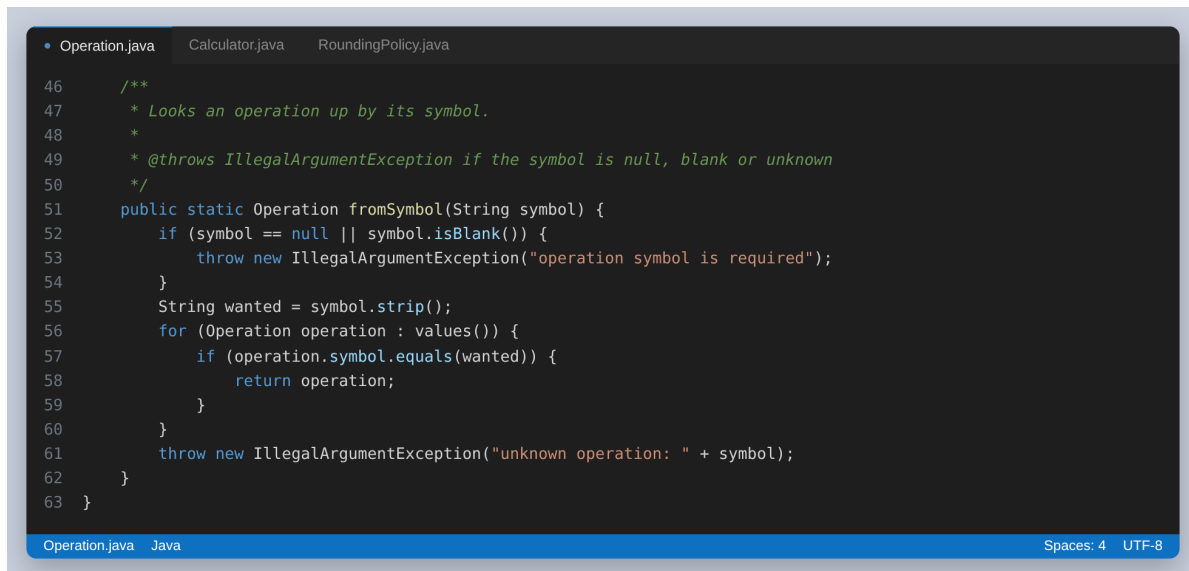


```
18 public enum Operation {
19
20     ADD("+", (a, b) -> a + b),
21     SUBTRACT("-", (a, b) -> a - b),
22     MULTIPLY("*", (a, b) -> a * b),
23     DIVIDE("/", (a, b) -> a / b);
24
25     private final String symbol;
26     private final DoubleBinaryOperator arithmetic;
27
28     Operation(String symbol, DoubleBinaryOperator arithmetic) {
29         this.symbol = symbol;
30         this.arithmetic = arithmetic;
31     }
}
```

Figure 2: The four operations, each with its symbol and its raw arithmetic

A second reason for the enum is that it gives the tests a list they cannot fall behind. A parameterized test over `Operation.values()` covers a fifth operation the day somebody adds one, without anybody remembering to update the test.

The symbol lookup is the one piece of parsing in the project, and it is strict about what it will not accept. Null, blank and unknown symbols all produce the same refusal, which gives the parameterized tests something to check that is not arithmetic.

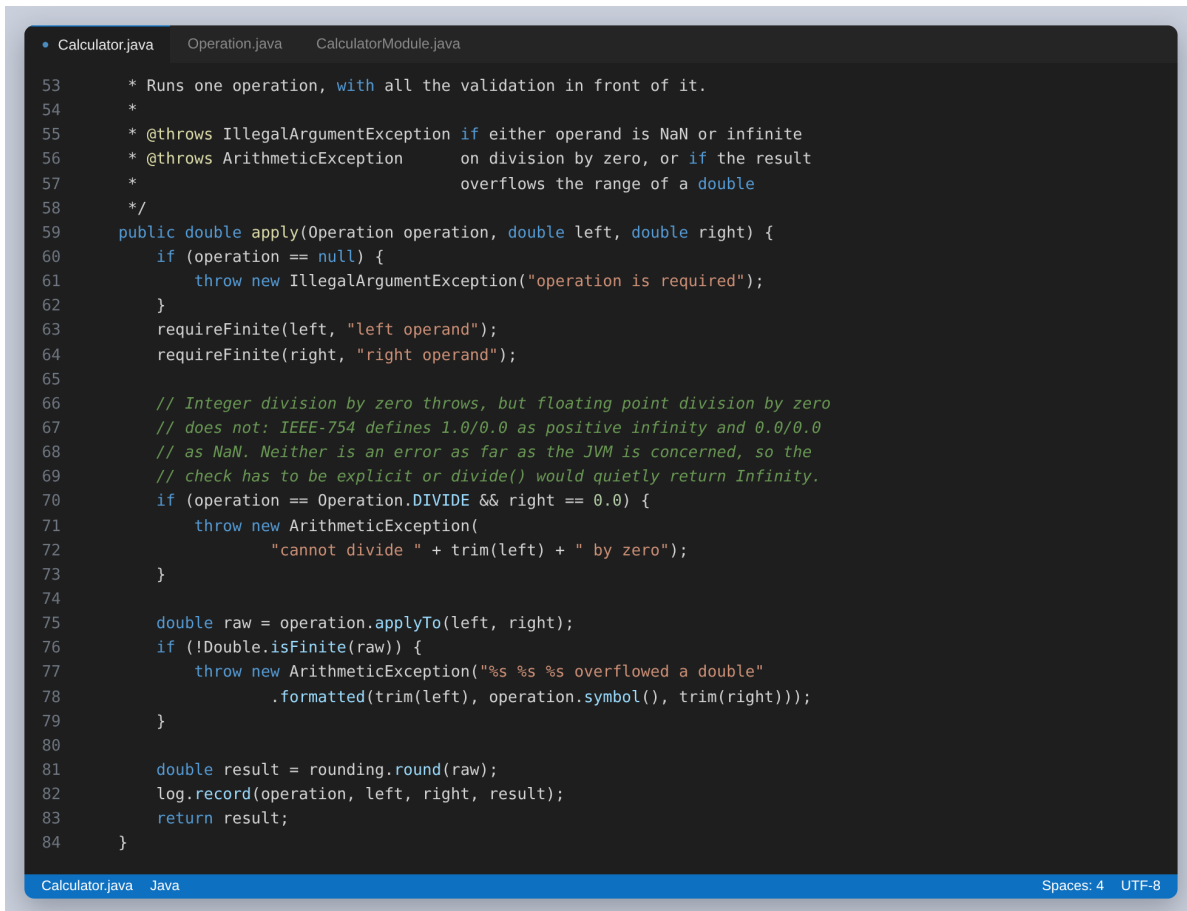
A screenshot of an IDE window showing a Java file named Operation.java. The code defines a static method fromSymbol that takes a String symbol and returns an Operation. It includes a Javadoc comment and handles three failure cases: null symbol, blank symbol, and unknown symbol. The IDE interface shows tabs for Operation.java, Calculator.java, and RoundingPolicy.java. The status bar at the bottom indicates 'Operation.java Java' and 'Spaces: 4 UTF-8'.

```
46  /**
47   * Looks an operation up by its symbol.
48   *
49   * @throws IllegalArgumentException if the symbol is null, blank or unknown
50   */
51  public static Operation fromSymbol(String symbol) {
52      if (symbol == null || symbol.isBlank()) {
53          throw new IllegalArgumentException("operation symbol is required");
54      }
55      String wanted = symbol.strip();
56      for (Operation operation : values()) {
57          if (operation.symbol.equals(wanted)) {
58              return operation;
59          }
60      }
61      throw new IllegalArgumentException("unknown operation: " + symbol);
62  }
63 }
```

Figure 3: Looking an operation up by symbol, and the three ways that can fail

The one method that matters

Everything the calculator enforces is in apply. Three rules, and the first is the one worth explaining.



```
53  * Runs one operation, with all the validation in front of it.
54  *
55  * @throws IllegalArgumentException if either operand is NaN or infinite
56  * @throws ArithmeticException  on division by zero, or if the result
57  *                               overflows the range of a double
58  */
59  public double apply(Operation operation, double left, double right) {
60      if (operation == null) {
61          throw new IllegalArgumentException("operation is required");
62      }
63      requireFinite(left, "left operand");
64      requireFinite(right, "right operand");
65
66      // Integer division by zero throws, but floating point division by zero
67      // does not: IEEE-754 defines 1.0/0.0 as positive infinity and 0.0/0.0
68      // as NaN. Neither is an error as far as the JVM is concerned, so the
69      // check has to be explicit or divide() would quietly return Infinity.
70      if (operation == Operation.DIVIDE && right == 0.0) {
71          throw new ArithmeticException(
72              "cannot divide " + trim(left) + " by zero");
73      }
74
75      double raw = operation.applyTo(left, right);
76      if (!Double.isFinite(raw)) {
77          throw new ArithmeticException("%s %s %s overflowed a double"
78              .formatted(trim(left), operation.symbol(), trim(right)));
79      }
80
81      double result = rounding.round(raw);
82      log.record(operation, left, right, result);
83      return result;
84  }
```

Figure 4: All four operations funnel through this method, so the validation cannot be avoided

Integer division by zero throws `ArithmeticException`, so it is easy to assume floating point division does too. It does not. IEEE-754 defines 1.0 divided by 0.0 as positive infinity and 0.0 divided by 0.0 as NaN, and the JVM treats neither as an error. Without the explicit check, divide would quietly return Infinity and every test expecting an exception would fail for a reason that looks like a test bug rather than a design gap.

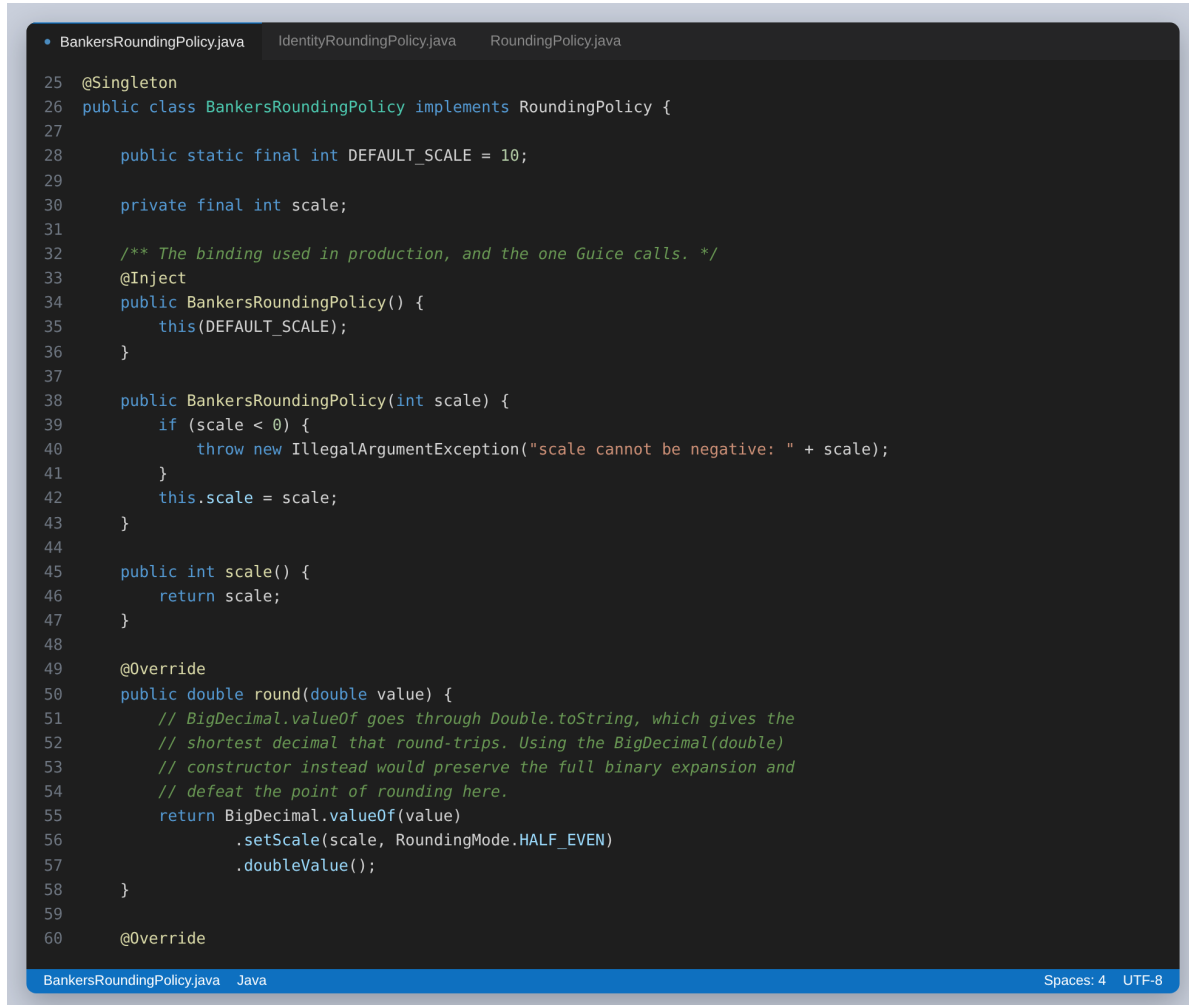
The overflow check is the same idea one step further out. Multiplying two large but perfectly valid numbers produces infinity, and returning that would push the problem onto the caller. The calculator treats a result it cannot represent as an error, and says which calculation caused it.

The third rule is that a NaN or infinite operand is rejected before any arithmetic happens. That one is about error messages: NaN propagates silently through every operation, so a calculation that started with one produces a NaN result and no clue where it came from.

The two collaborators, and why they exist

The calculator is given a rounding policy and an operation log through its constructor, and constructs neither. That is the whole of the dependency injection design; the containers later on are just different ways of calling this constructor.

Rounding is a policy rather than a line of code because of the oldest problem in floating point arithmetic. 0.1 and 0.2 have no exact binary representation, so their sum is 0.30000000000000004. The default policy rounds to ten decimal places, which removes the noise at the end of the mantissa without touching any digit a calculation actually cares about.



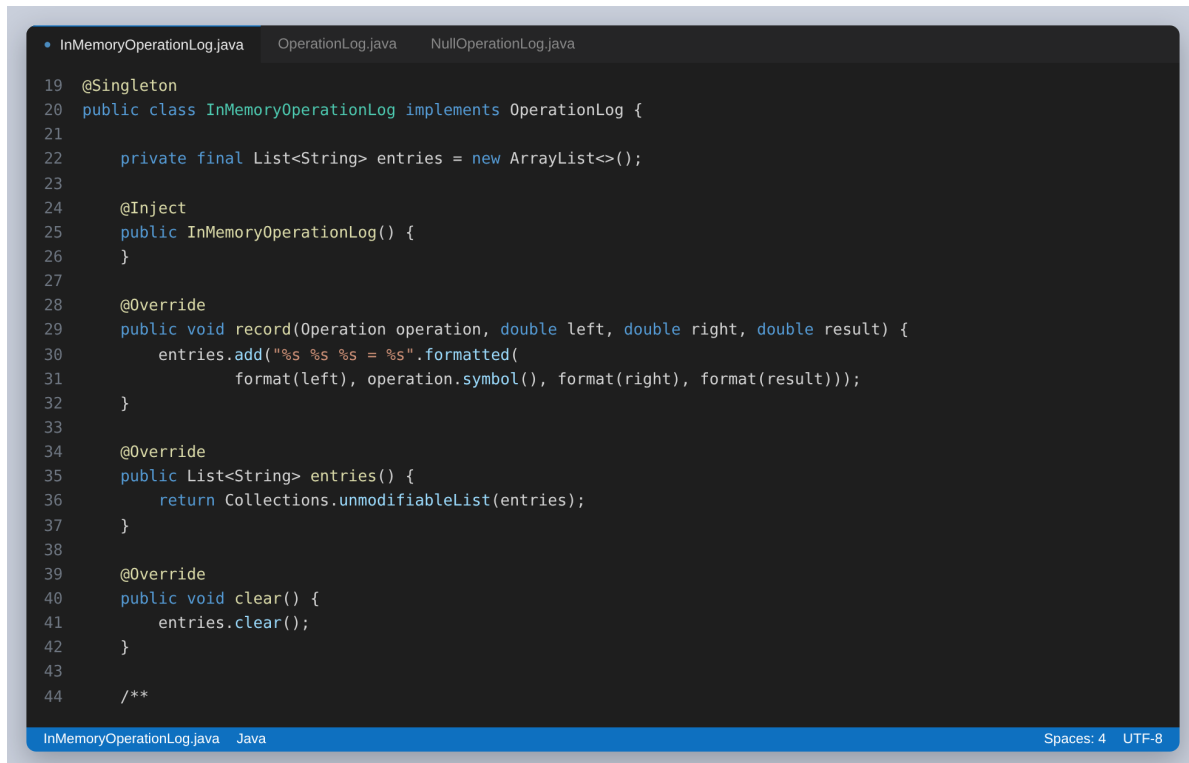
```
25 @Singleton
26 public class BankersRoundingPolicy implements RoundingPolicy {
27
28     public static final int DEFAULT_SCALE = 10;
29
30     private final int scale;
31
32     /** The binding used in production, and the one Guice calls. */
33     @Inject
34     public BankersRoundingPolicy() {
35         this(DEFAULT_SCALE);
36     }
37
38     public BankersRoundingPolicy(int scale) {
39         if (scale < 0) {
40             throw new IllegalArgumentException("scale cannot be negative: " + scale);
41         }
42         this.scale = scale;
43     }
44
45     public int scale() {
46         return scale;
47     }
48
49     @Override
50     public double round(double value) {
51         // BigDecimal.valueOf goes through Double.toString, which gives the
52         // shortest decimal that round-trips. Using the BigDecimal(double)
53         // constructor instead would preserve the full binary expansion and
54         // defeat the point of rounding here.
55         return BigDecimal.valueOf(value)
56             .setScale(scale, RoundingMode.HALF_EVEN)
57             .doubleValue();
58     }
59
60     @Override
```

Figure 5: Banker's rounding, and the reason the scale is ten

HALF_EVEN is the other half of that class. It sends a tie to the nearest even digit rather than always away from zero, so a long run of rounded values does not drift upwards. There is a parameterized test further on that pins this down against the HALF_UP behaviour most people expect.

The second policy, IdentityRoundingPolicy, returns the value untouched. It exists so the tests can show the raw result, and it is the more useful of the two for explaining anything.

The log is there to give the tests an observable side effect. Without it every test could only check a return value, there would be nothing for an @AfterEach to clean up, and the lifecycle annotations would look decorative. It is bound as a singleton, which is exactly what makes the cleanup necessary.



```
19 @Singleton
20 public class InMemoryOperationLog implements OperationLog {
21
22     private final List<String> entries = new ArrayList<>();
23
24     @Inject
25     public InMemoryOperationLog() {
26     }
27
28     @Override
29     public void record(Operation operation, double left, double right, double result) {
30         entries.add("%s %s %s = %s".formatted(
31             format(left), operation.symbol(), format(right), format(result)));
32     }
33
34     @Override
35     public List<String> entries() {
36         return Collections.unmodifiableList(entries);
37     }
38
39     @Override
40     public void clear() {
41         entries.clear();
42     }
43
44     /**
```

InMemoryOperationLog.java Java Spaces: 4 UTF-8

Figure 6: The log the tests assert on. Bound as a singleton, so it carries state between tests unless something clears it.

Running the application shows all of it at once: the wiring Guice produced, the four operations, the same sum under both rounding policies, and the four things it refuses.

```
bash - the calculator running

$ mvn -q compile exec:java
Calculator, wired by Guice
  rounding : BankersRoundingPolicy(scale=10)
  log      : InMemoryOperationLog

The four operations
4 + 5      = 9.0
10 - 3     = 7.0
6 * 7      = 42.0
9 / 4      = 2.25

Rounding, which is a policy rather than a hard-coded line
0.1 + 0.2  = 0.3    (rounded to 10 places)
0.1 + 0.2  = 0.30000000000000004  (what the hardware computed)

What it refuses
1 / 0      ArithmeticException: cannot divide 1 by zero
MAX_VALUE * 2  ArithmeticException: 1.7976931348623157E308 * 2 overflowed a double
NaN + 1      IllegalArgumentException: left operand is not a number
operation '^' IllegalArgumentException: unknown operation: ^

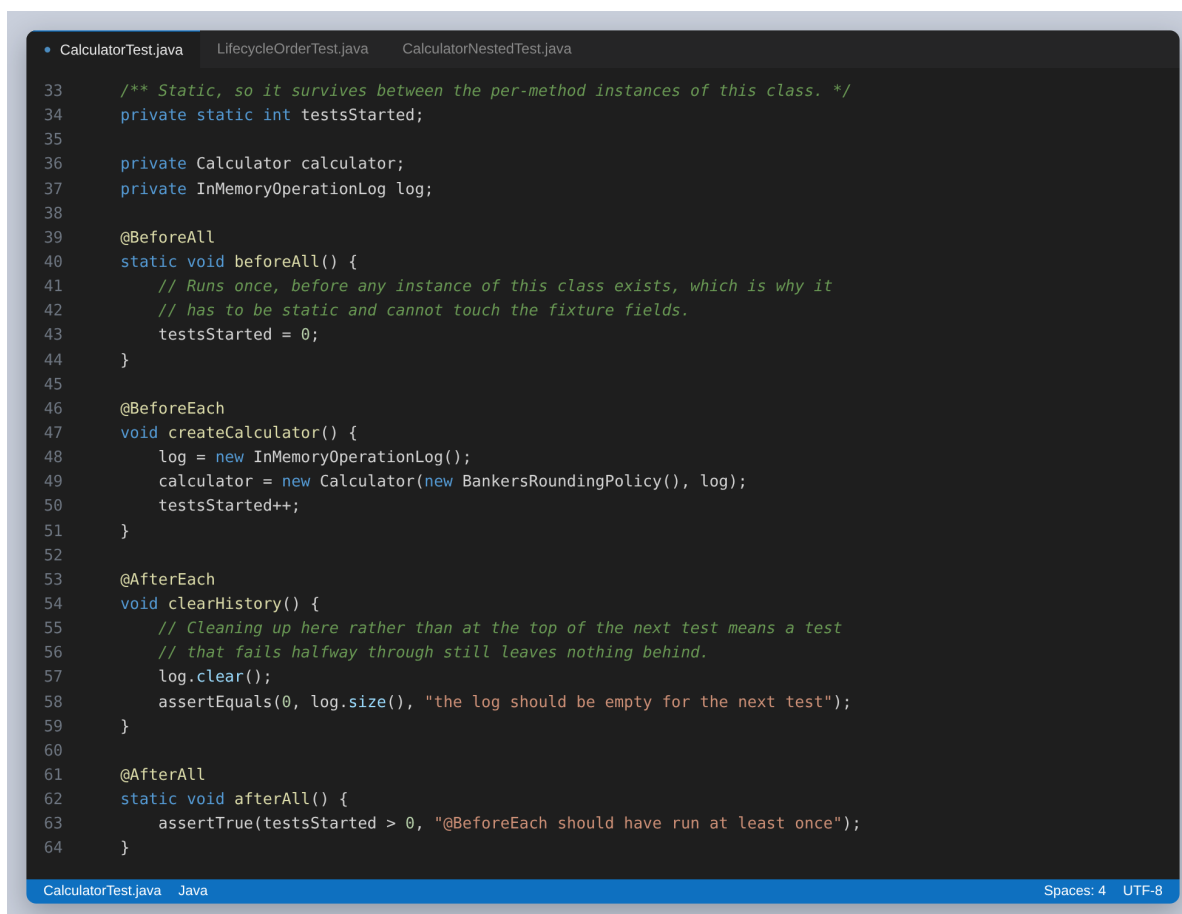
History
4 + 5 = 9
10 - 3 = 7
6 * 7 = 42
9 / 4 = 2.25
0.1 + 0.2 = 0.3
```

Figure 7: The calculator run from the command line

Step 2: Annotations and assertions

CalculatorTest is the class the assignment describes: the lifecycle annotations, assertEquals and assertsThrows, applied to the four operations. Seventeen tests.

The fixture is rebuilt in @BeforeEach rather than in a field initialiser, and the log is cleared in @AfterEach. Both matter here in a way they would not if the calculator were stateless.

The image shows a screenshot of an IDE with three tabs: CalculatorTest.java, LifecycleOrderTest.java, and CalculatorNestedTest.java. The CalculatorTest.java tab is active, showing the following code:

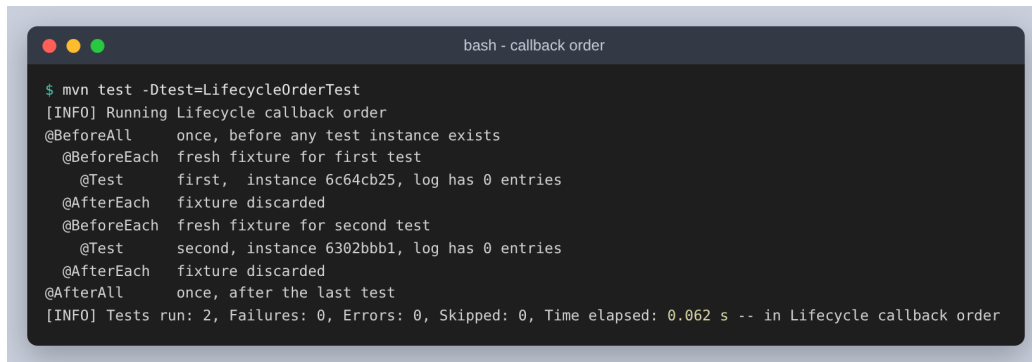
```
33  /** Static, so it survives between the per-method instances of this class. */
34  private static int testsStarted;
35
36  private Calculator calculator;
37  private InMemoryOperationLog log;
38
39  @BeforeAll
40  static void beforeAll() {
41      // Runs once, before any instance of this class exists, which is why it
42      // has to be static and cannot touch the fixture fields.
43      testsStarted = 0;
44  }
45
46  @BeforeEach
47  void createCalculator() {
48      log = new InMemoryOperationLog();
49      calculator = new Calculator(new BankersRoundingPolicy(), log);
50      testsStarted++;
51  }
52
53  @AfterEach
54  void clearHistory() {
55      // Cleaning up here rather than at the top of the next test means a test
56      // that fails halfway through still leaves nothing behind.
57      log.clear();
58      assertEquals(0, log.size(), "the log should be empty for the next test");
59  }
60
61  @AfterAll
62  static void afterAll() {
63      assertTrue(testsStarted > 0, "@BeforeEach should have run at least once");
64  }
```

The status bar at the bottom indicates "CalculatorTest.java Java" and "Spaces: 4 UTF-8".

Figure 8: The four lifecycle callbacks. @BeforeAll and @AfterAll have to be static, because they run before and after any instance exists.

Clearing in @AfterEach rather than at the start of the next test is a small decision with a real consequence: a test that fails halfway through still leaves nothing behind for the next one.

What actually keeps the tests independent, though, is not the @BeforeEach. JUnit constructs a new instance of the test class for every @Test method. LifecycleOrderTest prints the identity hash of the instance alongside each callback, and the two hashes are different.

A terminal window titled "bash - callback order" showing the output of a Maven test command. The output displays the lifecycle of two test instances, including @BeforeAll, @BeforeEach, @Test, @AfterEach, and @AfterAll annotations, along with fixture creation and discarding messages. The tests pass successfully.

```
$ mvn test -Dtest=LifecycleOrderTest
[INFO] Running Lifecycle callback order
@BeforeAll    once, before any test instance exists
@BeforeEach   fresh fixture for first test
@Test         first, instance 6c64cb25, log has 0 entries
@AfterEach    fixture discarded
@BeforeEach   fresh fixture for second test
@Test         second, instance 6302bbb1, log has 0 entries
@AfterEach    fixture discarded
@AfterAll     once, after the last test
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.062 s -- in Lifecycle callback order
```

Figure 9: The callback order, printed by the tests themselves. Two tests, two instances, and a fresh fixture for each.

Assertions

The assignment names `assertEquals` and `assertThrows`. Both are used throughout, along with four others where they say something the first two cannot.

Assertion	Used for
<code>assertEquals(expected, actual)</code>	an exact result, where the value is exactly representable
<code>assertEquals(expected, actual, delta)</code>	a double whose last bits are not meaningful
<code>assertThrows(type, executable)</code>	a refusal, returning the exception so the message can be checked
<code>assertThrowsExactly(type, executable)</code>	a refusal where a subclass would not do
<code>assertAll(executables...)</code>	several independent checks, all reported together
<code>assertNotEquals</code> , <code>assertSame</code> , <code>assertNotSame</code>	identity, and the floating point comparison that does not hold

`assertThrows` is the one with a trap in it. The call under test has to happen inside the lambda; written outside it, the exception escapes before the assertion can catch it and the test errors instead of passing. Capturing the return value and asserting on the message is also worth the extra line, because a test that only checks the type would still pass if the calculator threw for an unrelated reason.

```
CalculatorTest.java LifecycleOrderTest.java CalculatorNestedTest.java

109     @Test
110     @DisplayName("dividing by zero throws ArithmeticException")
111     void divideByZeroThrows() {
112         // The lambda is the important part: assertThrows needs the call to
113         // happen inside it, otherwise the exception escapes before the
114         // assertion can catch it.
115         ArithmeticException thrown = assertThrows(ArithmeticException.class,
116             () -> calculator.divide(1, 0));
117
118         // Asserting on the message as well. A test that only checks the type
119         // would still pass if the calculator threw for some unrelated reason.
120         assertEquals("cannot divide 1 by zero", thrown.getMessage());
121     }
122
123     @Test
124     @DisplayName("0 / 0 is refused too, rather than returning NaN")
125     void zeroDividedByZeroThrows() {
126         assertThrows(ArithmeticException.class, () -> calculator.divide(0, 0));
127     }
```

Figure 10: Two refusals. The second one exists because 0.0 divided by 0.0 is NaN rather than infinity, and both have to be refused.

assertAll is the assertion that changed how the arithmetic tests are written. Four separate assertEquals calls stop at the first failure, so a calculator with three broken operations takes three runs to diagnose. assertAll evaluates every executable and reports them together.

```
CalculatorTest.java LifecycleOrderTest.java CalculatorNestedTest.java

94     @Test
95     @DisplayName("all four at once, so one failure does not hide the rest")
96     void allFourOperations() {
97         // assertAll evaluates every executable even after one has failed, and
98         // reports them together. Four separate assertEquals calls would stop at
99         // the first, which turns fixing a broken calculator into four runs.
100         assertAll("the four operations",
101             () -> assertEquals(9.0, calculator.add(4, 5), "add"),
102             () -> assertEquals(7.0, calculator.subtract(10, 3), "subtract"),
103             () -> assertEquals(42.0, calculator.multiply(6, 7), "multiply"),
104             () -> assertEquals(2.25, calculator.divide(9, 4), "divide"));
105     }
```

Figure 11: One failure does not hide the other three

The assertion that is almost always wrong

assertEquals on two doubles compares them exactly. For 9.0 that is fine. For anything that came out of a decimal fraction it is a test that will fail on a value that is correct.

```
CalculatorTest.java LifecycleOrderTest.java FloatingPointFailureDemo.java

218 @Test
219 @DisplayName("without the policy the same sum is 0.30000000000000004")
220 void withoutRoundingTheErrorIsVisible() {
221     Calculator raw = new Calculator(new IdentityRoundingPolicy(), new NullOperationLog());
222
223     // The exact comparison that a beginner writes, and that fails.
224     assertEquals(0.3, raw.add(0.1, 0.2));
225
226     // Two assertions that do hold. The three-argument assertEquals takes a
227     // tolerance, and is the right assertion for a double whose last bits
228     // are not meaningful.
229     assertEquals(0.3, raw.add(0.1, 0.2), 1e-9);
230     assertEquals(0.30000000000000004, raw.add(0.1, 0.2));
231 }
232 }
```

CalculatorTest.java Java Spaces: 4 UTF-8

Figure 12: The exact comparison that fails, and the two that hold

FloatingPointFailureDemo contains three tests that fail on purpose, so the failure output can be shown rather than described. It is excluded from every normal run by the pom and has its own profile.

```
FloatingPointFailureDemo.java CalculatorTest.java IdentityRoundingPolicy.java

44 @Test
45 @DisplayName("assertAll reports every failure it finds, not just the first")
46 void everyFailureIsReported() {
47     // Four assertions, three of which fail. The one that passes is the
48     // interesting one: 0.1 * 0.3 is exactly 0.03 as a double, while
49     // 0.3 - 0.2 is 0.09999999999999998 and 1.1 * 1.1 is 1.2100000000000002.
50     //
51     // Representation error is not a rule that applies to every decimal, it
52     // depends on the particular values and the particular operation. That is
53     // the reason to use a tolerance everywhere rather than only where a
54     // failure has already been seen – the cases that happen to be exact
55     // today give no assurance about the next ones.
56     assertAll(
57         () -> assertEquals(0.3, raw.add(0.1, 0.2), "add: 0.1 + 0.2"),
58         () -> assertEquals(0.1, raw.subtract(0.3, 0.2), "subtract: 0.3 - 0.2"),
59         () -> assertEquals(0.03, raw.multiply(0.1, 0.3), "multiply: 0.1 * 0.3"),
60         () -> assertEquals(1.21, raw.multiply(1.1, 1.1), "multiply: 1.1 * 1.1"));
61 }
```

FloatingPointFailureDemo.java Java Spaces: 4 UTF-8

Figure 13: Four assertions, three of which fail. The one that passes is the interesting one.

```
bash - mvn test -Pshow-failure

$ mvn test -Pshow-failure
[INFO] Running Deliberate failures
[ERROR] Tests run: 3, Failures: 3, Errors: 0, Skipped: 0, Time elapsed: 0.067 s <<< FAILURE! -- in Deliberate failures
[ERROR] com.techstart.calculator.FloatingPointFailureDemo.withAMessage -- Time elapsed: 0.034 s <<< FAILURE!
org.opentest4j.AssertionFailedError: no exact binary form for 0.1 or 0.2; compare with a tolerance ==> expected: <0.3> but was: <0.30000000000000004>
    at com.techstart.calculator.FloatingPointFailureDemo.withAMessage(FloatingPointFailureDemo.java:70)

[ERROR] com.techstart.calculator.FloatingPointFailureDemo.everyFailureIsReported -- Time elapsed: 0.007 s <<< FAILURE!
org.opentest4j.MultipleFailuresError:
Multiple Failures (3 failures)
    org.opentest4j.AssertionFailedError: add: 0.1 + 0.2 ==> expected: <0.3> but was: <0.30000000000000004>
    org.opentest4j.AssertionFailedError: subtract: 0.3 - 0.2 ==> expected: <0.1> but was: <0.09999999999999998>
    org.opentest4j.AssertionFailedError: multiply: 1.1 * 1.1 ==> expected: <1.21> but was: <1.2100000000000002>
    at com.techstart.calculator.FloatingPointFailureDemo.everyFailureIsReported(FloatingPointFailureDemo.java:56)
[ERROR] com.techstart.calculator.FloatingPointFailureDemo.exactEqualityOnADouble -- Time elapsed: 0.002 s <<< FAILURE!
org.opentest4j.AssertionFailedError: expected: <0.3> but was: <0.30000000000000004>
    at com.techstart.calculator.FloatingPointFailureDemo.exactEqualityOnADouble(FloatingPointFailureDemo.java:41)
```

Figure 14: The same three failures as JUnit reports them

The multiply assertion on 0.1 and 0.3 passes, and it is absent from the list of failures for that reason: 0.1 multiplied by 0.3 is exactly 0.03 as a double, while 0.3 minus 0.2 is 0.09999999999999998 and 1.1 multiplied by 1.1 is 1.2100000000000002. Representation error is not a rule that applies to every decimal; it depends on the particular values and the particular operation. That is the argument for using a tolerance everywhere rather than only where a failure has already been seen, because the cases that happen to be exact today say nothing about the next ones.

The middle failure also shows what `assertAll` adds. All three failing assertions are listed under one `Multiple Failures` heading, from a single test, rather than the first one stopping the method.

Step 3: Parameterized tests

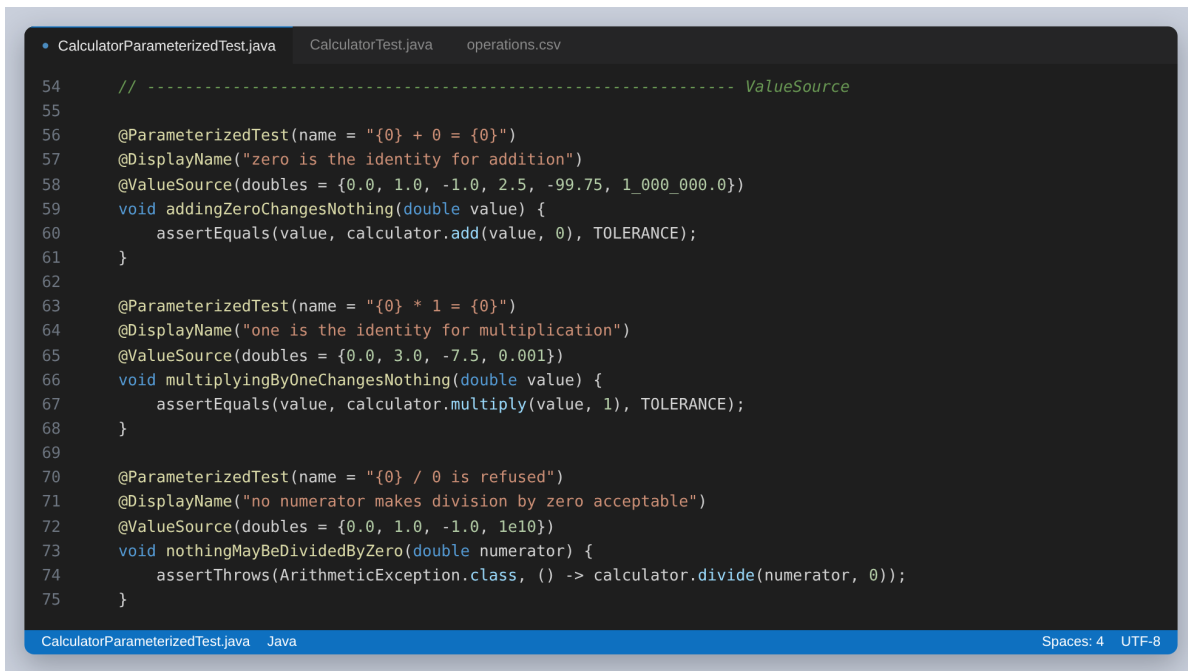
Sixty-six of the 185 tests come from eleven parameterized methods. Six argument sources are used, each because it is the right tool for a different job rather than to work through the API.

Source	Used for	Cases
@ValueSource	one value changing at a time	14
@CsvSource	operands and an expected result, next to the test	8
@CsvFileSource	the same table in a file that can grow	10
@MethodSource	cases that are computed rather than typed	16
@EnumSource	every operation, and it cannot fall behind	10
@NullAndEmptySource	the two inputs everybody forgets	8

@BeforeEach runs before every generated case, not once per method, so each row still gets a clean calculator. This is the difference from a @TestFactory later on, where it does not.

@ValueSource, for one changing value

The simplest source, and the right one for a property that involves a single operand: zero is the identity for addition, one is the identity for multiplication, and no numerator makes division by zero acceptable.



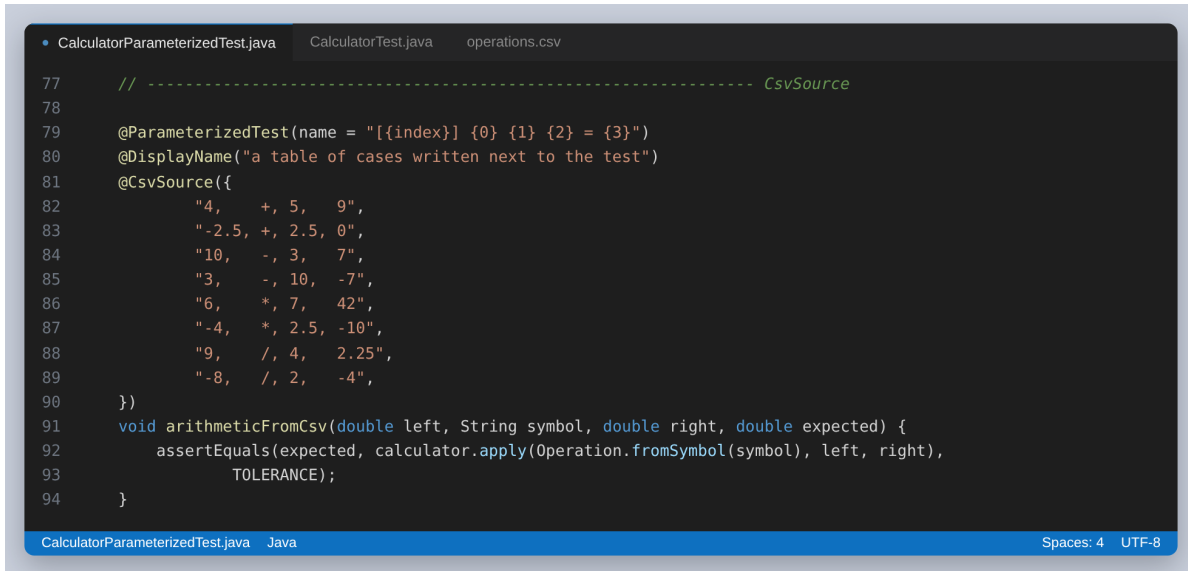
```
CalculatorParameterizedTest.java CalculatorTest.java operations.csv

54 // ----- ValueSource
55
56 @ParameterizedTest(name = "{0} + 0 = {0}")
57 @DisplayName("zero is the identity for addition")
58 @ValueSource(doubles = {0.0, 1.0, -1.0, 2.5, -99.75, 1_000_000.0})
59 void addingZeroChangesNothing(double value) {
60     assertEquals(value, calculator.add(value, 0), TOLERANCE);
61 }
62
63 @ParameterizedTest(name = "{0} * 1 = {0}")
64 @DisplayName("one is the identity for multiplication")
65 @ValueSource(doubles = {0.0, 3.0, -7.5, 0.001})
66 void multiplyingByOneChangesNothing(double value) {
67     assertEquals(value, calculator.multiply(value, 1), TOLERANCE);
68 }
69
70 @ParameterizedTest(name = "{0} / 0 is refused")
71 @DisplayName("no numerator makes division by zero acceptable")
72 @ValueSource(doubles = {0.0, 1.0, -1.0, 1e10})
73 void nothingMayBeDividedByZero(double numerator) {
74     assertThrows(ArithmeticException.class, () -> calculator.divide(numerator, 0));
75 }
```

Figure 15: Three properties, fourteen cases, and no expected values to maintain

@CsvSource and @CsvFileSource, for a table

Once a case needs operands and an expected result, a CSV row is the clearest way to write it. The name pattern is worth setting, because the default names cases by index and a report full of case 7 is no help when one of them fails.



```
CalculatorParameterizedTest.java CalculatorTest.java operations.csv

77 // ----- CsvSource
78
79 @ParameterizedTest(name = "{index} {0} {1} {2} = {3}")
80 @DisplayName("a table of cases written next to the test")
81 @CsvSource({
82     "4, +, 5, 9",
83     "-2.5, +, 2.5, 0",
84     "10, -, 3, 7",
85     "3, -, 10, -7",
86     "6, *, 7, 42",
87     "-4, *, 2.5, -10",
88     "9, /, 4, 2.25",
89     "-8, /, 2, -4",
90 })
91 void arithmeticFromCsv(double left, String symbol, double right, double expected) {
92     assertEquals(expected, calculator.apply(Operation.fromSymbol(symbol), left, right),
93         TOLERANCE);
94 }
```

Figure 16: A table written next to the test, aligned so it reads as a table

The same table moved into a file, so it can grow without the test class growing. The file is the interesting part of this pairing: it is read twice in this project, once by the annotation here and once by a dynamic test later.

```
left, symbol, right, expected
4, +, 5, 9
-2.5, +, 2.5, 0
0.1, +, 0.2, 0.3
10, -, 3, 7
3, -, 10, -7
6, *, 7, 42
-4, *, 2.5, -10
9, /, 4, 2.25
-8, /, 2, -4
100, /, 3, 33.3333333333
```

```
96 // ----- CsvFileSource
97
98 @ParameterizedTest(name = "[{index}] {0} {1} {2} = {3}")
99 @DisplayName("the same table, loaded from src/test/resources/operations.csv")
100 @CsvFileSource(resources = "/operations.csv", numLinesToSkip = 1)
101 void arithmeticFromCsvFile(double left, String symbol, double right, double expected) {
102     assertEquals(expected, calculator.apply(Operation.fromSymbol(symbol), left, right),
103         TOLERANCE);
104 }
```

Figure 17: numLinesToSkip is what skips the header row

Both of the last two rows are there for a reason. The 0.1 plus 0.2 row passes only because the rounding policy is applied, and the 100 divided by 3 row pins down what rounding to ten places actually produces.

```
$ java -jar junit-platform-console-standalone.jar execute -m 'CalculatorParameterizedTest#unusableSymbols(java.lang.String)' ...
└─ JUnit Jupiter ✓
  └─ Calculator: parameterized ✓
    └─ the same table, loaded from src/test/resources/operations.csv ✓
      └─ [1] 4 + 5 = 9 ✓
      └─ [2] -2.5 + 2.5 = 0 ✓
      └─ [3] 0.1 + 0.2 = 0.3 ✓
      └─ [4] 10 - 3 = 7 ✓
      └─ [5] 3 - 10 = -7 ✓
      └─ [6] 6 * 7 = 42 ✓
      └─ [7] -4 * 2.5 = -10 ✓
      └─ [8] 9 / 4 = 2.25 ✓
      └─ [9] -8 / 2 = -4 ✓
      └─ [10] 100 / 3 = 33.3333333333 ✓
    └─ null, empty and blank symbols are all refused ✓
      └─ symbol [null] is refused ✓
      └─ symbol [] is refused ✓
      └─ symbol [ ] is refused ✓
      └─ symbol [ ] is refused ✓
      └─ symbol [^] is refused ✓
      └─ symbol [**] is refused ✓
      └─ symbol [plus] is refused ✓
      └─ symbol [++] is refused ✓
```

Figure 18: The ten rows of the file, and the eight unusable symbols, each reported as its own test

@MethodSource, for cases that have to be computed

A factory method can produce cases that would be tedious or impossible to type. The first of these generates twelve powers of two and checks doubling against addition to itself; the second is the one that pins down banker's rounding.

```
CalculatorParameterizedTest.java CalculatorTest.java operations.csv

106 // ----- MethodSource
107
108 /**
109  * Cases that would be tedious to type: powers of two, checked against a
110  * doubling that does not use the calculator.
111  */
112 static Stream<Arguments> doublingCases() {
113     return Stream.iterate(1.0, value -> value * 2)
114         .limit(12)
115         .map(value -> Arguments.of(value, value + value));
116 }
117
118 @ParameterizedTest(name = "{0} * 2 = {1}")
119 @DisplayName("generated cases, from a factory method")
120 @MethodSource("doublingCases")
121 void multiplyingByTwoIsAdditionToItself(double value, double expected) {
122     assertEquals(expected, calculator.multiply(value, 2), TOLERANCE);
123 }
124
125 static Stream<Arguments> roundingCases() {
126     // Ties, which is where HALF_EVEN and HALF_UP disagree. At scale 1:
127     // 0.25 -> 0.2 (2 is even) and 0.35 -> 0.4 (4 is even). HALF_UP would
128     // give 0.3 and 0.4.
129     return Stream.of(
130         Arguments.of(0.25, 0.2),
131         Arguments.of(0.35, 0.4),
132         Arguments.of(0.45, 0.4),
133         Arguments.of(0.55, 0.6));
134 }
135
136 @ParameterizedTest(name = "{0} rounds to {1} at one decimal place")
137 @DisplayName("banker's rounding sends a tie to the nearest even digit")
138 @MethodSource("roundingCases")
139 void banksRoundingBreaksTiesTowardsEven(double raw, double expected) {
140     Calculator oneDecimalPlace = new Calculator(
141         new BankersRoundingPolicy(1), new NullOperationLog());
142     assertEquals(expected, oneDecimalPlace.add(raw, 0), TOLERANCE);
143 }

CalculatorParameterizedTest.java Java Spaces: 4 UTF-8
```

Figure 19: Generated cases, and the four ties where HALF_EVEN and HALF_UP disagree

The rounding cases are the useful ones. At one decimal place 0.25 rounds to 0.2 and 0.35 rounds to 0.4, because both go to the nearest even digit. HALF_UP would give 0.3 and 0.4. Four rows are enough to make the difference between the two modes a fact about the code rather than a claim in a comment.

@EnumSource, for a list that maintains itself

Three properties that should hold for every operation, whatever the operation is. Nothing here restates the list, so a fifth operation would be covered by all three the moment it is added to the enum.

```

• CalculatorParameterizedTest.java  CalculatorTest.java  operations.csv

145 // ----- EnumSource
146
147 @ParameterizedTest(name = "{0}")
148 @DisplayName("every operation records exactly one history entry")
149 @EnumSource(Operation.class)
150 void everyOperationIsLogged(Operation operation) {
151     // Whatever the operation, 6 and 3 are safe operands for it.
152     calculator.apply(operation, 6, 3);
153     assertEquals(1, calculator.history().size());
154     assertTrue(calculator.history().entries().get(0).contains(operation.symbol()),
155         () -> "expected the " + operation.symbol() + " symbol in "
156             + calculator.history().entries());
157 }
158
159 @ParameterizedTest(name = "{0} has a symbol that round-trips")
160 @DisplayName("every symbol can be looked back up")
161 @EnumSource(Operation.class)
162 void symbolsRoundTrip(Operation operation) {
163     assertEquals(operation, Operation.fromSymbol(operation.symbol()));
164     // And with surrounding whitespace, which is what a parsed input has.
165     assertEquals(operation, Operation.fromSymbol(" " + operation.symbol() + " "));
166 }
167
168 @ParameterizedTest(name = "{0} and {1} are not the same operation")
169 @DisplayName("the two operations that are not commutative")
170 @EnumSource(names = {"SUBTRACT", "DIVIDE"})
171 void nonCommutativeOperations(Operation operation) {
172     assertTrue(calculator.apply(operation, 10, 2)
173         != calculator.apply(operation, 2, 10));
174 }

```

CalculatorParameterizedTest.java Java Spaces: 4 UTF-8

Figure 20: Every operation, and the two that are not commutative selected by name

```

bash - @EnumSource, @MethodSource, @ValueSource

$ java -jar junit-platform-console-standalone.jar execute -m 'CalculatorParameterizedTest#everyOperationIsLogged(...)' ...

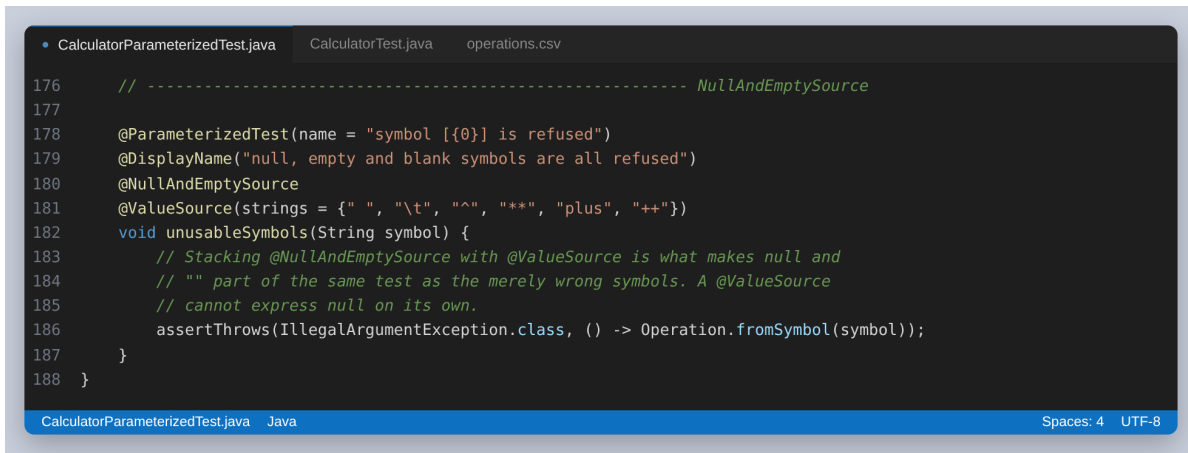
└─ JUnit Jupiter ✓
  └─ Calculator: parameterized ✓
    └─ every operation records exactly one history entry ✓
      └─ ADD ✓
      └─ SUBTRACT ✓
      └─ MULTIPLY ✓
      └─ DIVIDE ✓
    └─ banker's rounding sends a tie to the nearest even digit ✓
      └─ 0.25 rounds to 0.2 at one decimal place ✓
      └─ 0.35 rounds to 0.4 at one decimal place ✓
      └─ 0.45 rounds to 0.4 at one decimal place ✓
      └─ 0.55 rounds to 0.6 at one decimal place ✓
    └─ zero is the identity for addition ✓
      └─ 0.0 + 0 = 0.0 ✓
      └─ 1.0 + 0 = 1.0 ✓
      └─ -1.0 + 0 = -1.0 ✓
      └─ 2.5 + 0 = 2.5 ✓
      └─ -99.75 + 0 = -99.75 ✓
      └─ 1000000.0 + 0 = 1000000.0 ✓

```

Figure 21: @EnumSource, @MethodSource and @ValueSource in the report

@NullAndEmptySource, for the two inputs everybody forgets

A `@ValueSource` cannot express null. Stacking `@NullAndEmptySource` on top of one is what puts null and the empty string into the same test as the merely wrong symbols, which is where they belong: they are all inputs the lookup has to refuse.

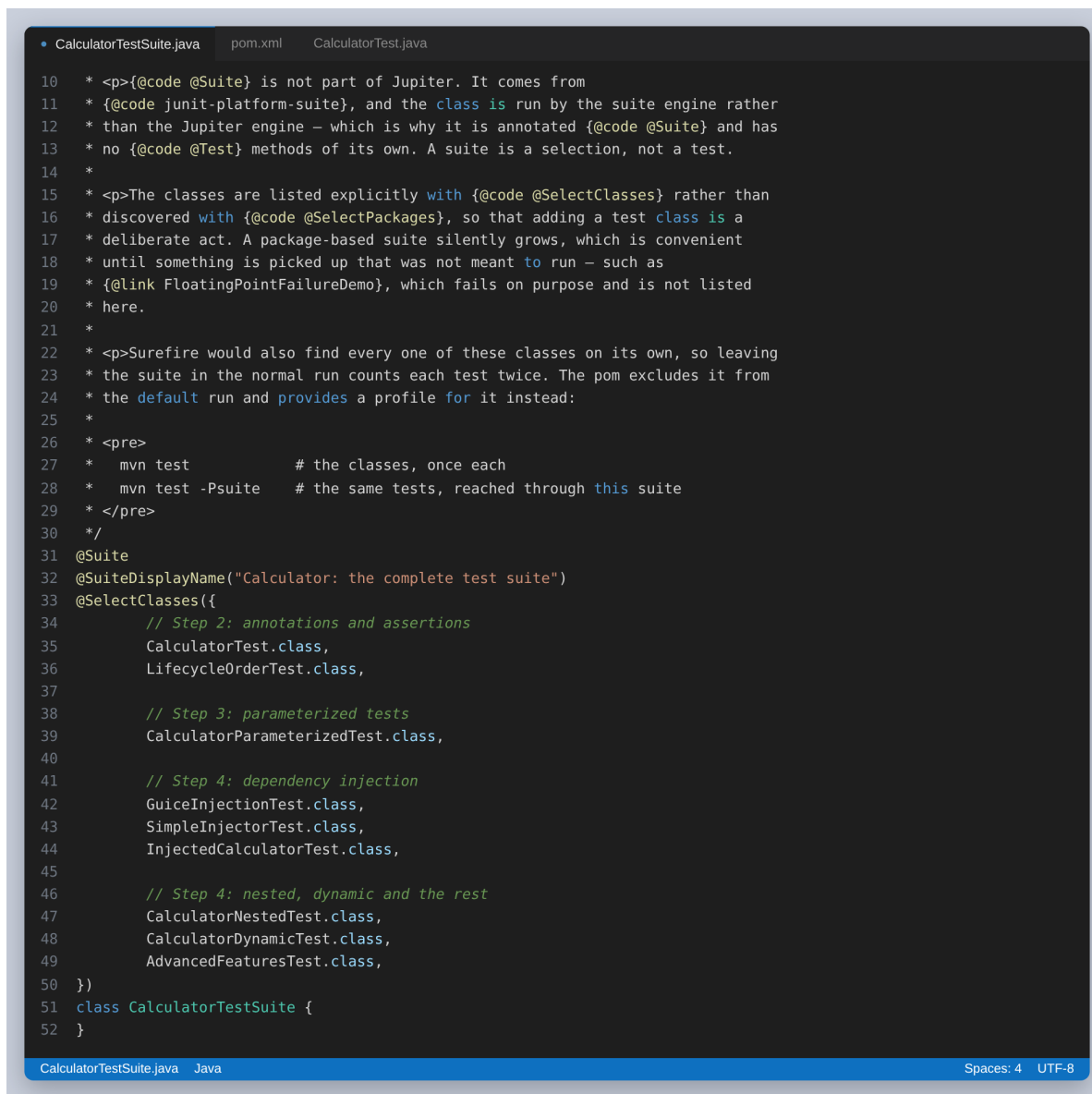


```
176 // ----- NullAndEmptySource
177
178 @ParameterizedTest(name = "symbol [{0}] is refused")
179 @DisplayName("null, empty and blank symbols are all refused")
180 @NullAndEmptySource
181 @ValueSource(strings = {" ", "\t", "^", "*", "plus", "++"})
182 void unusableSymbols(String symbol) {
183     // Stacking @NullAndEmptySource with @ValueSource is what makes null and
184     // "" part of the same test as the merely wrong symbols. A @ValueSource
185     // cannot express null on its own.
186     assertThrows(IllegalArgumentException.class, () -> Operation.fromSymbol(symbol));
187 }
188 }
```

Figure 22: Eight unusable symbols, including the two that cannot be written as a `@ValueSource`

Step 3, continued: The test suite

The suite gathers all nine test classes. It is worth being precise about what it is: `@Suite` comes from `junit-platform-suite`, not from Jupiter, and the class is run by the suite engine. It has no `@Test` methods of its own, because a suite is a selection rather than a test.



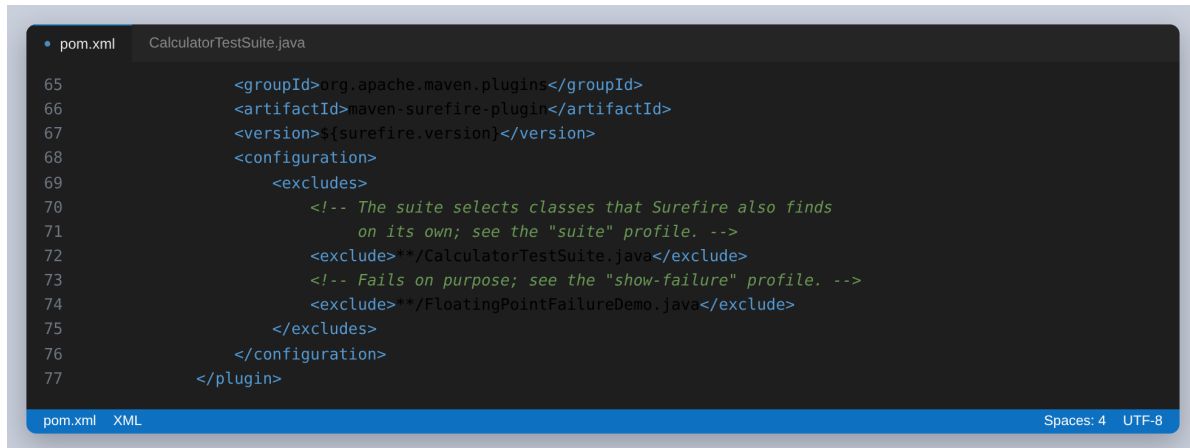
```
10 * <p>{@code @Suite} is not part of Jupiter. It comes from
11 * {@code junit-platform-suite}, and the class is run by the suite engine rather
12 * than the Jupiter engine – which is why it is annotated {@code @Suite} and has
13 * no {@code @Test} methods of its own. A suite is a selection, not a test.
14 *
15 * <p>The classes are listed explicitly with {@code @SelectClasses} rather than
16 * discovered with {@code @SelectPackages}, so that adding a test class is a
17 * deliberate act. A package-based suite silently grows, which is convenient
18 * until something is picked up that was not meant to run – such as
19 * {@link FloatingPointFailureDemo}, which fails on purpose and is not listed
20 * here.
21 *
22 * <p>Surefire would also find every one of these classes on its own, so leaving
23 * the suite in the normal run counts each test twice. The pom excludes it from
24 * the default run and provides a profile for it instead:
25 *
26 * <pre>
27 * mvn test          # the classes, once each
28 * mvn test -Psuite  # the same tests, reached through this suite
29 * </pre>
30 */
31 @Suite
32 @SuiteDisplayName("Calculator: the complete test suite")
33 @SelectClasses({
34     // Step 2: annotations and assertions
35     CalculatorTest.class,
36     LifecycleOrderTest.class,
37
38     // Step 3: parameterized tests
39     CalculatorParameterizedTest.class,
40
41     // Step 4: dependency injection
42     GuiceInjectionTest.class,
43     SimpleInjectorTest.class,
44     InjectedCalculatorTest.class,
45
46     // Step 4: nested, dynamic and the rest
47     CalculatorNestedTest.class,
48     CalculatorDynamicTest.class,
49     AdvancedFeaturesTest.class,
50 })
51 class CalculatorTestSuite {
52 }
```

Figure 23: Nine classes, listed explicitly and grouped by the step they belong to

The classes are listed with `@SelectClasses` rather than discovered with `@SelectPackages`, so adding a test class to the suite is a deliberate act. A package-based suite silently grows, which is convenient until it picks up something that was not meant to run. This project has exactly such a class: `FloatingPointFailureDemo` fails on purpose, and it is not in the list.

The problem the suite creates

Surefire finds every one of those nine classes on its own. Leaving the suite in the default run means each test is discovered twice, once directly and once through the suite, and the reported total doubles. The suite is excluded from the default run and given a profile of its own instead.



```
65     <groupId>org.apache.maven.plugins</groupId>
66     <artifactId>maven-surefire-plugin</artifactId>
67     <version>${surefire.version}</version>
68     <configuration>
69         <excludes>
70             <!-- The suite selects classes that Surefire also finds
71                  on its own; see the "suite" profile. -->
72             <exclude>**/CalculatorTestSuite.java</exclude>
73             <!-- Fails on purpose; see the "show-failure" profile. -->
74             <exclude>**/FloatingPointFailureDemo.java</exclude>
75         </excludes>
76     </configuration>
77 </plugin>
```

CalculatorTestSuite.java

pom.xml XML Spaces: 4 UTF-8

Figure 24: The two exclusions, and why each is there

```
• pom.xml CalculatorTestSuite.java

90 <profiles>
91   <!-- mvn test -Psuite runs the aggregated suite and nothing else. -->
92   <profile>
93     <id>suite</id>
94     <build>
95       <plugins>
96         <plugin>
97           <groupId>org.apache.maven.plugins</groupId>
98           <artifactId>maven-surefire-plugin</artifactId>
99           <configuration combine.self="override">
100             <includes>
101               <include>**/CalculatorTestSuite.java</include>
102             </includes>
103           </configuration>
104         </plugin>
105       </plugins>
106     </build>
107   </profile>
108
109   <!-- mvn test -Pshow-failure keeps the deliberately failing test, which
110        is excluded from every other run. -->
111   <profile>
112     <id>show-failure</id>
113     <build>
114       <plugins>
115         <plugin>
116           <groupId>org.apache.maven.plugins</groupId>
117           <artifactId>maven-surefire-plugin</artifactId>
118           <configuration combine.self="override">
119             <includes>
120               <include>**/FloatingPointFailureDemo.java</include>
121             </includes>
122             <testFailureIgnore>true</testFailureIgnore>
123           </configuration>
124         </plugin>
125       </plugins>
126     </build>
127   </profile>
128 </profiles>
```

pom.xml XML Spaces: 4 UTF-8

Figure 25: Two profiles: one to run the suite, one to see the deliberate failures

```
mvn test           # the nine classes, once each: 185 tests
mvn test -Psuite   # the same 185, reached through the suite
mvn test -Pshow-failure # only the three deliberate failures
```

Running through the suite reports the same 185 tests, which is the point: the suite changes how the tests are reached, not what runs.

```
bash - mvn test -Psuite

$ mvn test -Psuite
[INFO] Running com.techstart.calculator.CalculatorTestSuite
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.103 s -- in Calculator: the four arithmetic operations
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.023 s -- in Lifecycle callback order
[INFO] Tests run: 66, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.184 s -- in Calculator: parameterized
[INFO] Running compared with the hand-rolled injector
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in compared with the hand-rolled injector
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in when a test overrides part of the module
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in refusing
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s -- in resolving
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.040 s -- in Dependency injection by JUnit's ParameterResolver
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in when the log is discarded
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in when the rounding policy is swapped out
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in when dividing
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in when adding
[INFO] Tests run: 40, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.057 s -- in Calculator: dynamic tests
[INFO] Tests run: 185, Failures: 0, Errors: 0, Skipped: 2
[INFO] BUILD SUCCESS
```

Figure 26: The suite run. The same total as the default run.

The console launcher shows the shape of it more clearly than Surefire does. The suite engine sits above the Jupiter engine, which then reports the nine classes underneath. The full tree is 245 lines; this is the same run with only the class level kept.

```
bash - the suite, collapsed to the class level

$ java -jar junit-platform-console-standalone.jar execute -c CalculatorTestSuite
└─ JUnit Platform Suite ✓
  └─ Calculator: the complete test suite ✓
    └─ JUnit Jupiter ✓
      └─ Calculator: the four arithmetic operations ✓
      └─ Lifecycle callback order ✓
      └─ Calculator: parameterized ✓
      └─ Dependency injection with Guice ✓
      └─ The hand-rolled SimpleInjector ✓
      └─ Dependency injection by JUnit's ParameterResolver ✓
      └─ A calculator ✓
      └─ Calculator: dynamic tests ✓
      └─ Advanced JUnit 5 features ✓

Test run finished after 610 ms
[ 42 containers found ]
[ 0 containers skipped ]
[ 42 containers started ]
[ 0 containers aborted ]
[ 42 containers successful ]
[ 0 containers failed ]
[ 185 tests found ]
[ 2 tests skipped ]
[ 183 tests started ]
[ 0 tests aborted ]
[ 183 tests successful ]
[ 0 tests failed ]
```

Figure 27: The suite engine, the Jupiter engine, and the nine classes

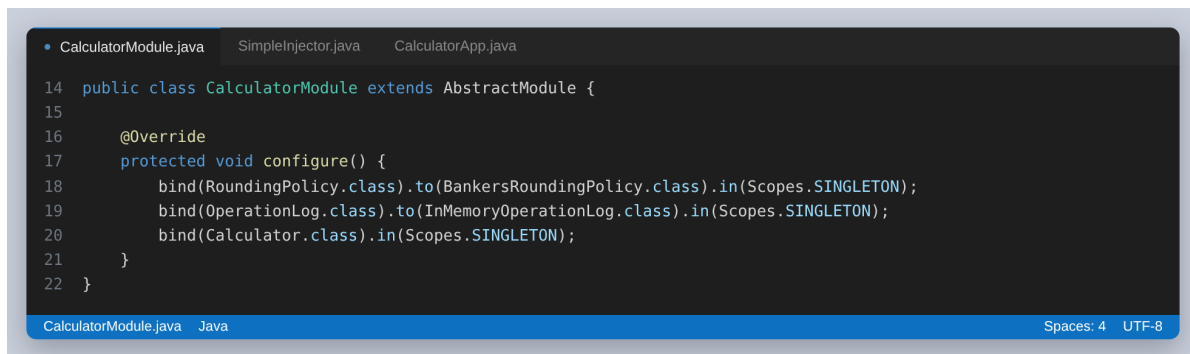
Step 4: Dependency injection

The assignment allows either a simple injector or a framework. This project has both, because writing the small one is what makes the framework's job concrete, and there are then two things to compare.

There is also a third kind of injection in the project, and it is the one specific to JUnit 5: injecting into the tests themselves rather than into the application.

Guice

Three bindings. The point of them is that they are the only place the choice of implementation is written down.

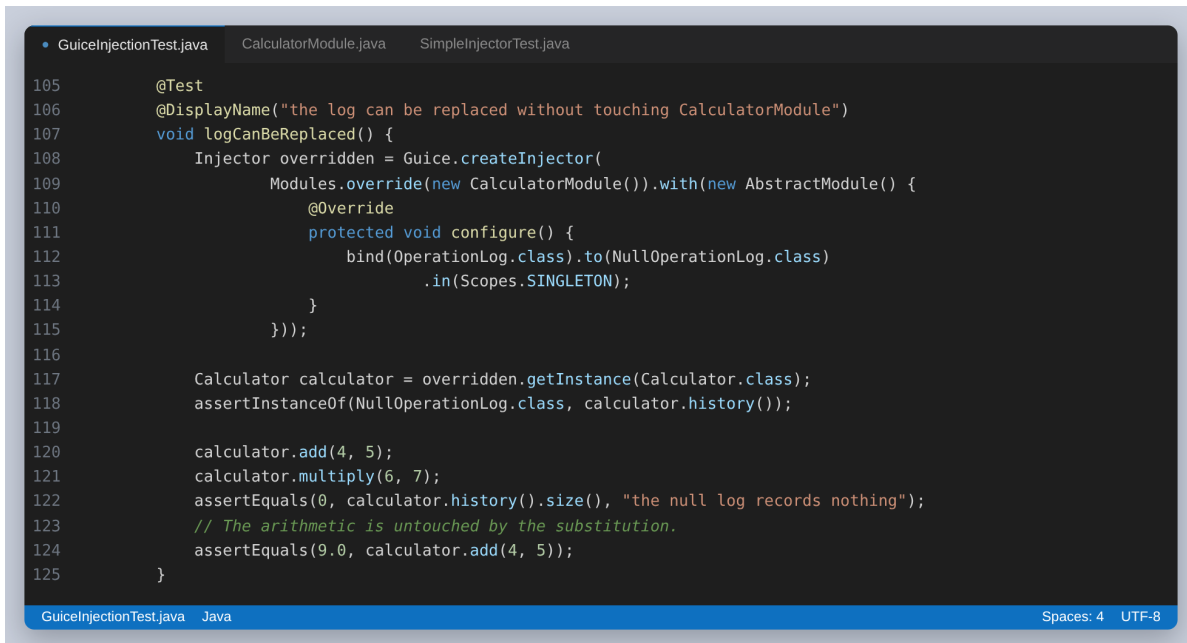


```
14 public class CalculatorModule extends AbstractModule {
15
16     @Override
17     protected void configure() {
18         bind(RoundingPolicy.class).to(BankersRoundingPolicy.class).in(Scopes.SINGLETON);
19         bind(OperationLog.class).to(InMemoryOperationLog.class).in(Scopes.SINGLETON);
20         bind(Calculator.class).in(Scopes.SINGLETON);
21     }
22 }
```

The screenshot shows a code editor with three tabs: CalculatorModule.java, SimpleInjector.java, and CalculatorApp.java. The CalculatorModule.java tab is active, showing the code above. The status bar at the bottom indicates 'CalculatorModule.java Java' and 'Spaces: 4 UTF-8'.

Figure 28: The production wiring, in three lines

What makes that worth having is what a test can then do without editing it. `Modules.override` replaces one binding and leaves the rest alone, so a test can swap the log for one that discards everything, or the rounding policy for one that does nothing, and show that the arithmetic is unaffected either way.



```
105     @Test
106     @DisplayName("the log can be replaced without touching CalculatorModule")
107     void logCanBeReplaced() {
108         Injector overridden = Guice.createInjector(
109             Modules.override(new CalculatorModule()).with(new AbstractModule() {
110                 @Override
111                 protected void configure() {
112                     bind(OperationLog.class).to(NullOperationLog.class)
113                         .in(Scopes.SINGLETON);
114                 }
115             });
116
117         Calculator calculator = overridden.getInstance(Calculator.class);
118         assertNotNull(NullOperationLog.class, calculator.history());
119
120         calculator.add(4, 5);
121         calculator.multiply(6, 7);
122         assertEquals(0, calculator.history().size(), "the null log records nothing");
123         // The arithmetic is untouched by the substitution.
124         assertEquals(9.0, calculator.add(4, 5));
125     }
```

Figure 29: Replacing one binding without touching the module

A fresh injector is built in `@BeforeEach` rather than shared across the class. Sharing one would share the singletons inside it, and the operation log would leak from test to test. The tests check that too: two injectors produce calculators whose histories are independent.

The hand-rolled injector

Eighty lines, and the idea turns out to be small. Keep a map from interface to implementation, find the constructor to call, resolve each of its parameters the same way, and cache the result if the type is a singleton.

```
SimpleInjector.java CalculatorModule.java Calculator.java

47 private <T> T resolve(Class<T> requested, Set<Class<?>> inProgress) {
48     Object existing = singletons.get(requested);
49     if (existing != null) {
50         return requested.cast(existing);
51     }
52
53     Class<?> target = bindings.getOrDefault(requested, requested);
54
55     if (target.isInterface()) {
56         throw new IllegalStateException("no binding for " + target.getName());
57     }
58     if (!inProgress.add(target)) {
59         throw new IllegalStateException("dependency cycle: " + describe(inProgress, target));
60     }
61
62     Constructor<?> constructor = chooseConstructor(target);
63     Object[] arguments = new Object[constructor.getParameterCount()];
64     Class<?>[] parameterTypes = constructor.getParameterTypes();
65     for (int i = 0; i < arguments.length; i++) {
66         arguments[i] = resolve(parameterTypes[i], inProgress);
67     }
68
69     Object created;
70     try {
71         constructor.setAccessible(true);
72         created = constructor.newInstance(arguments);
73     } catch (InstantiationException | IllegalAccessException e) {
74         throw new IllegalStateException("could not construct " + target.getName(), e);
75     } catch (InvocationTargetException e) {
76         // Surface what the constructor threw, not the reflection wrapper.
77         throw new IllegalStateException(
78             target.getSimpleName() + " constructor failed", e.getCause());
79     }
80
81     inProgress.remove(target);
82
83     if (target.isAnnotationPresent(jakarta.inject.Singleton.class)) {
84         singletons.put(requested, created);
85         singletons.put(target, created);
86     }
87     return requested.cast(created);
88 }
89
90 /**
```

SimpleInjector.java Java Spaces: 4 UTF-8

Figure 30: The whole of resolution, including the cycle check

Choosing the constructor is where the design decision is. Guessing at the longest one is how a container ends up building something the author did not intend, so this injector takes the constructor annotated `@Inject`, or the only one there is, and otherwise refuses. A clear failure at wiring time is cheaper than a puzzling object later.

```
• SimpleInjector.java  CalculatorModule.java  Calculator.java

89  /**
90   * The constructor annotated {@code @Inject}, or the only one there is.
91   *
92   * <p>Being strict here is deliberate. Guessing at the longest constructor
93   * is how a container ends up doing something the author did not intend, and
94   * a clear failure at wiring time is cheaper than a puzzling object later.
95   */
96  private static Constructor<?> chooseConstructor(Class<?> target) {
97      Constructor<?>[] all = target.getDeclaredConstructors();
98      Constructor<?> annotated = null;
99      for (Constructor<?> candidate : all) {
100         if (candidate.isAnnotationPresent(Inject.class)) {
101             if (annotated != null) {
102                 throw new IllegalStateException(
103                     target.getName() + " has more than one @Inject constructor");
104             }
105             annotated = candidate;
106         }
107     }
108     if (annotated != null) {
109         return annotated;
110     }
111     if (all.length == 1) {
112         return all[0];
113     }
114     throw new IllegalStateException(
115         target.getName() + " has " + all.length
116         + " constructors and none is annotated @Inject");
117 }
```

SimpleInjector.java Java Spaces: 4 UTF-8

Figure 31: Strict on purpose: no @Inject and more than one constructor is an error

Writing it meant designing its failures as well as its successes, and six of the twelve tests in SimpleInjectorTest are about those: an unbound interface, a missing binding further down the graph, a dependency cycle reported with the chain that caused it, two kinds of ambiguous constructor, and a constructor that throws, where the cause reported is what the constructor threw rather than the reflection wrapper around it.

Both containers produce a calculator that behaves identically, and the tests assert that directly. What the small one does not do is the argument for the framework.

	SimpleInjector	Guice
Constructor injection	yes	yes
Singleton scope	yes	yes
Field and method injection	no	yes
Two bindings for one type, by qualifier	no	yes
Providers, for objects needing construction logic	no	yes
Overriding a module for a test	no	yes

	SimpleInjector	Guice
Cycle handling	reports it	reports or breaks it
Lines of code to maintain	about 80	none

Injection into the tests, by JUnit

This is the mechanism that has no equivalent in JUnit 4, and the reason a JUnit 4 test class is all fields and setUp methods. A test declares what it needs as a parameter and an extension provides it.

```

66     @Override
67     public Object resolveParameter(ParameterContext parameterContext,
68                                   ExtensionContext extensionContext) {
69         Calculator calculator = calculatorFor(extensionContext);
70         Class<?> type = parameterContext.getParameter().getType();
71
72         if (type == Calculator.class) {
73             return calculator;
74         }
75         if (type == OperationLog.class) {
76             // The same log the calculator writes to, so a test can be handed
77             // just the log and still be asserting on the right object.
78             return calculator.history();
79         }
80         throw new ParameterResolutionException("cannot resolve " + type.getName());
81     }
82
83     /**
84      * One calculator per test execution, or a throwaway one for a constructor.
85      *
86      * <p>Every parameter of a single test method resolves against the same
87      * context, so a method asking for both a calculator and its log gets a
88      * matching pair.
89      */
90     private Calculator calculatorFor(ExtensionContext context) {
91         if (context.getTestMethod().isEmpty()) {
92             // A class-level context, which means this is constructor injection.
93             // Storing here would share the value with every test in the class.
94             return new Calculator();
95         }
96         return context.getStore(NAMESPACE)
97             .getOrCreateIfAbsent(KEY, key -> new Calculator(), Calculator.class);
98     }

```

Figure 32: A ParameterResolver, and the store lookup that decides how long the value lives

InjectedCalculatorTest has no @BeforeEach and no fixture field at all. Every test that needs a calculator asks for one, and the cleanup that would have been an @AfterEach lives in the extension, so the guarantee is made once by the thing that owns the object instead of being restated in every test class.

```
• InjectedCalculatorTest.java CalculatorExtension.java GuiceInjectionTest.java

38     private final Calculator constructorInjected;
39
40     /**
41      * Constructor injection. JUnit resolves the parameter before it builds the
42      * instance, which means a test class can be immutable and have no
43      * uninitialised fields at all.
44      */
45     InjectedCalculatorTest(Calculator calculator) {
46         this.constructorInjected = calculator;
47     }
48
49     @Test
50     @DisplayName("the constructor was given a working calculator")
51     void constructorInjectionWorks() {
52         assertEquals(9.0, constructorInjected.add(4, 5));
53         assertTrue(constructorInjected.roundingPolicy() instanceof BankersRoundingPolicy);
54     }
55
56     @Test
57     @DisplayName("a method parameter is a different instance from the constructor's")
58     void methodInjectionDiffersFromConstructorInjection(Calculator injected) {
59         // Not an accident, and worth stating as a test so the behaviour is
60         // recorded rather than discovered again later.
61         //
62         // JUnit resolves a constructor parameter against the class-level
63         // ExtensionContext, before any test-scoped context exists. Because store
64         // lookups fall through to ancestor stores, an extension that cached the
65         // value there would hand the same calculator to every test in the class.
66         // CalculatorExtension therefore only caches under a test context, and
67         // gives a constructor its own instance.
68         assertNotSame(constructorInjected, injected);
69
70         // Both are fully wired, and neither has been used yet.
71         assertEquals(0, constructorInjected.history().size());
72         assertEquals(0, injected.history().size());
73     }

```

InjectedCalculatorTest.java Java Spaces: 4 UTF-8

Figure 33: Constructor injection, and the behaviour that had to be pinned down

What went wrong here, and what it taught me

The first version of the extension cached the calculator in the store of whatever context it was given. All fourteen tests passed. The isolation was gone anyway.

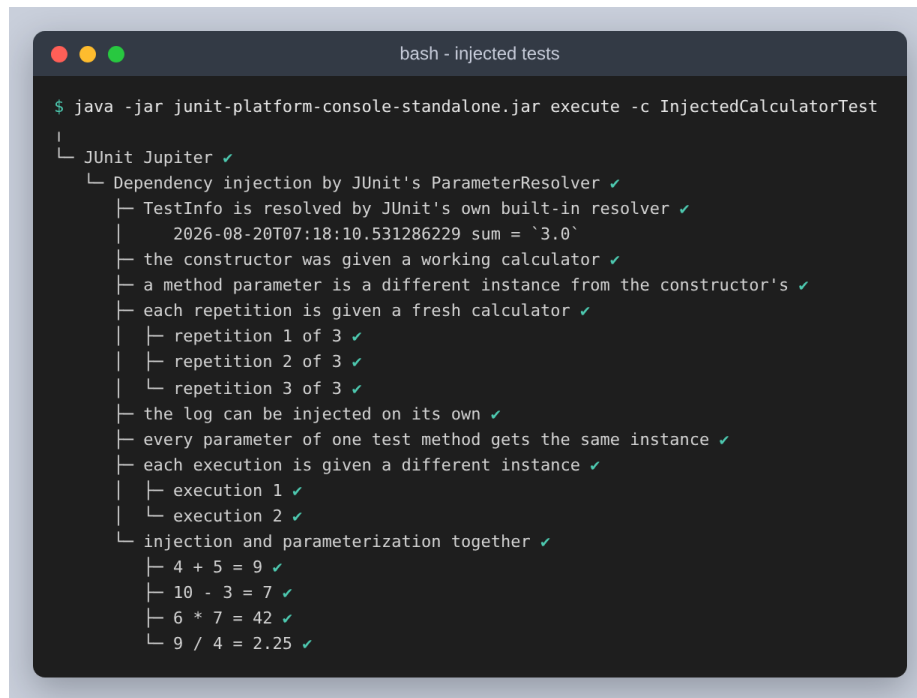
Printing the context during resolution showed why. JUnit resolves a constructor parameter against the class-level context, and a test method parameter against the method context:

```
[engine:junit-jupiter]/[class:InjectedCalculatorTest]
[engine:junit-jupiter]/[class:InjectedCalculatorTest]/[method:logInjected...]
```

Store lookups fall through to ancestor stores. Computing the value under the class-level context therefore put one calculator in the class store, and every test in the class then found that same object by inheritance. The tests still passed, because the extension's `afterEach` cleared the history between them, so nothing ever observed the sharing. The `@RepeatedTest` was the one that eventually caught it: three repetitions were being handed the same instance.

The fix is that the value is only stored when the context belongs to a test. A constructor parameter is given its own instance instead, which is still one calculator per test method, since the class is instantiated once per test. It is simply not the same one the method parameters get, and there is now a test asserting exactly that so the behaviour is recorded rather than rediscovered.

Making the two identical would have meant caching at class level, and per-test isolation is worth more than that consistency. The lesson is the one worth taking from the whole assignment: a passing suite is not the same as a correct one, and shared state is invisible precisely when something else is tidying up after it.



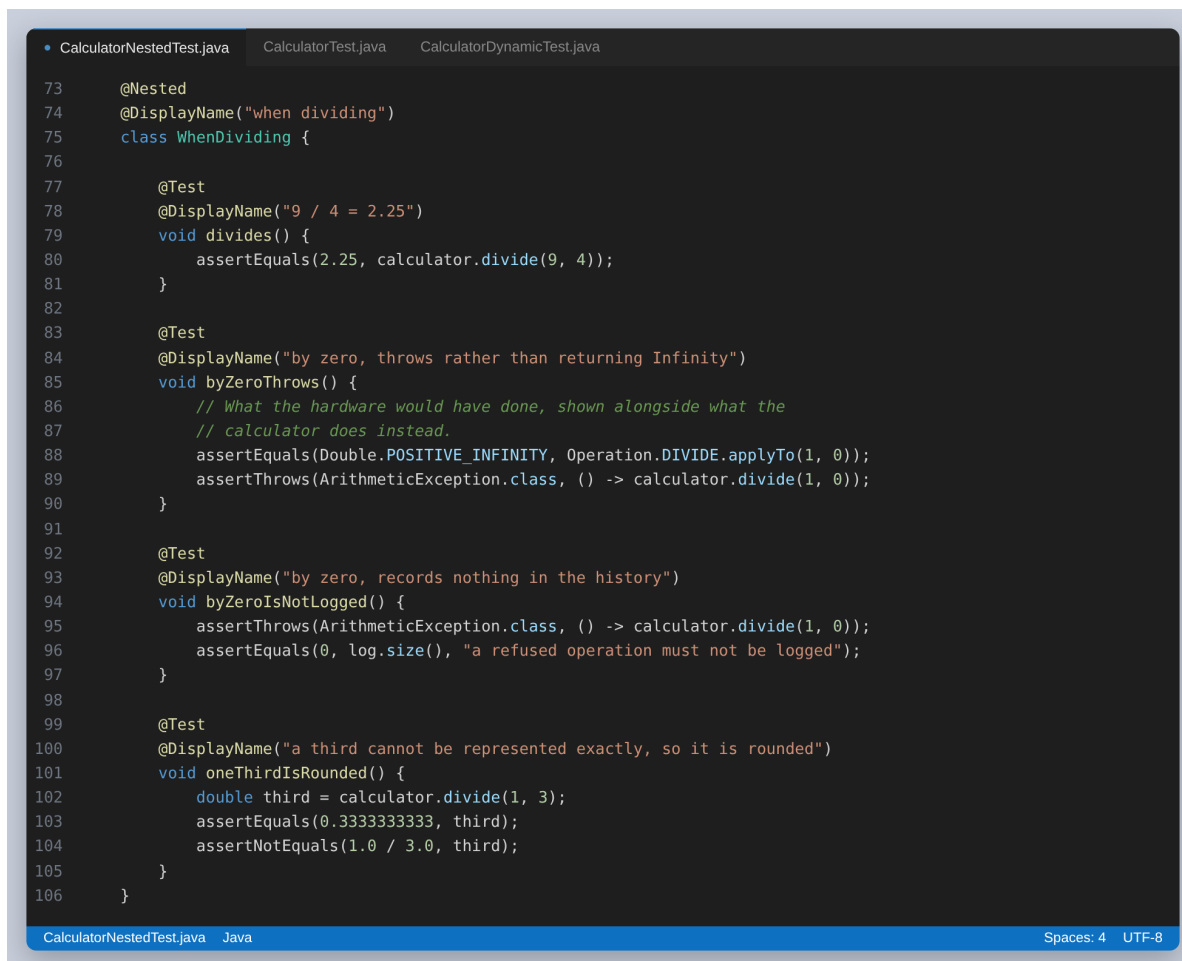
```
bash - injected tests

$ java -jar junit-platform-console-standalone.jar execute -c InjectedCalculatorTest
└─ JUnit Jupiter ✓
  └─ Dependency injection by JUnit's ParameterResolver ✓
    └─ TestInfo is resolved by JUnit's own built-in resolver ✓
      └─ 2026-08-20T07:18:10.531286229 sum = `3.0`
    └─ the constructor was given a working calculator ✓
    └─ a method parameter is a different instance from the constructor's ✓
    └─ each repetition is given a fresh calculator ✓
      └─ repetition 1 of 3 ✓
      └─ repetition 2 of 3 ✓
      └─ repetition 3 of 3 ✓
    └─ the log can be injected on its own ✓
    └─ every parameter of one test method gets the same instance ✓
    └─ each execution is given a different instance ✓
      └─ execution 1 ✓
      └─ execution 2 ✓
    └─ injection and parameterization together ✓
      └─ 4 + 5 = 9 ✓
      └─ 10 - 3 = 7 ✓
      └─ 6 * 7 = 42 ✓
      └─ 9 / 4 = 2.25 ✓
```

Figure 34: Fourteen tests with no fixture field between them. The published TestReporter entry appears under the test that wrote it.

Step 4, continued: Nested tests

@Nested groups tests by the situation they are about. The gain is not cosmetic: each inner class gets its own @BeforeEach, and JUnit runs the outer one first, so a group can add to the shared setup instead of repeating it.



```
73     @Nested
74     @DisplayName("when dividing")
75     class WhenDividing {
76
77         @Test
78         @DisplayName("9 / 4 = 2.25")
79         void divides() {
80             assertEquals(2.25, calculator.divide(9, 4));
81         }
82
83         @Test
84         @DisplayName("by zero, throws rather than returning Infinity")
85         void byZeroThrows() {
86             // What the hardware would have done, shown alongside what the
87             // calculator does instead.
88             assertEquals(Double.POSITIVE_INFINITY, Operation.DIVIDE.applyTo(1, 0));
89             assertThrows(ArithmeticException.class, () -> calculator.divide(1, 0));
90         }
91
92         @Test
93         @DisplayName("by zero, records nothing in the history")
94         void byZeroIsNotLogged() {
95             assertThrows(ArithmeticException.class, () -> calculator.divide(1, 0));
96             assertEquals(0, log.size(), "a refused operation must not be logged");
97         }
98
99         @Test
100        @DisplayName("a third cannot be represented exactly, so it is rounded")
101        void oneThirdIsRounded() {
102            double third = calculator.divide(1, 3);
103            assertEquals(0.3333333333, third);
104            assertNotEquals(1.0 / 3.0, third);
105        }
106    }
```

Figure 35: Everything true about division, and nothing else, in one group

The division group is the clearest example of what the grouping buys. Four tests, all about the same operation, one of which shows the raw infinity next to the exception the calculator throws instead. Read on its own, the group is a description of how division behaves.

A group that needs more than the shared fixture adds its own callback. This one builds a second calculator with the other rounding policy, which lets a single test compare the two.

```

CalculatorNestedTest.java CalculatorTest.java CalculatorDynamicTest.java
108     @Nested
109     @DisplayName("when the rounding policy is swapped out")
110     class WhenRoundingIsSwapped {
111
112         private Calculator raw;
113
114         @BeforeEach
115         void createRawCalculator() {
116             // This runs after the outer @BeforeEach, so both calculators exist
117             // and the two can be compared in one test.
118             raw = new Calculator(new IdentityRoundingPolicy(), new NullOperationLog());
119         }
120
121         @Test
122         @DisplayName("the outer @BeforeEach still ran first")
123         void bothFixturesExist() {
124             assertTrue(calculator.roundingPolicy() instanceof BankersRoundingPolicy);
125             assertTrue(raw.roundingPolicy() instanceof IdentityRoundingPolicy);
126         }
127
128         @Test
129         @DisplayName("the same sum gives two different answers")
130         void sameSumDifferentAnswer() {
131             assertEquals(0.3, calculator.add(0.1, 0.2));
132             assertEquals(0.30000000000000004, raw.add(0.1, 0.2));
133         }
134
135         @Test
136         @DisplayName("both are within a sensible tolerance of each other")
137         void bothAreCloseEnough() {
138             assertEquals(calculator.add(0.1, 0.2), raw.add(0.1, 0.2), 1e-9);
139         }
140     }

```

CalculatorNestedTest.java Java Spaces: 4 UTF-8

Figure 36: The inner `@BeforeEach` runs after the outer one, so both calculators exist

The inner classes have to be non-static. JUnit instantiates the outer class first and then the inner one against it, so a static nested class has no enclosing instance and its tests are not discovered at all.

The payoff is in the report, where the group names combine with the test names to read as sentences.

```
bash - @Nested

$ java -jar junit-platform-console-standalone.jar execute -c CalculatorNestedTest

└─ JUnit Jupiter ✓
  └─ A calculator ✓
    ├── starts with an empty history ✓
    ├── when the log is discarded ✓
    │   ├── the arithmetic is unchanged ✓
    │   ├── the empty history is always the same immutable list ✓
    │   └─ nothing is recorded ✓
    ├── when the rounding policy is swapped out ✓
    │   ├── the outer @BeforeEach still ran first ✓
    │   ├── both are within a sensible tolerance of each other ✓
    │   └─ the same sum gives two different answers ✓
    ├── when dividing ✓
    │   ├── by zero, records nothing in the history ✓
    │   ├── a third cannot be represented exactly, so it is rounded ✓
    │   ├── 9 / 4 = 2.25 ✓
    │   └─ by zero, throws rather than returning Infinity ✓
    └─ when adding ✓
        ├── is commutative ✓
        ├── uses the fixture built by the outer @BeforeEach ✓
        └─ 0.1 + 0.2 comes out as 0.3 because of the rounding policy ✓
```

Figure 37: Fourteen tests in four groups, and the outer test that belongs to none of them

Step 4, continued: Dynamic tests

A `@TestFactory` returns tests instead of being one. The difference from `@ParameterizedTest` is when the cases are decided: a parameterized test's cases are fixed at compile time by its annotation, while a factory's are whatever the code returns while the test is running.

	@ParameterizedTest	@TestFactory
Cases decided	at compile time	at run time
@BeforeEach runs	before every case	once, around the factory
Report shape	a flat list of cases	a tree, via <code>dynamicContainer</code>
Right for	a known set of inputs	a set that depends on something read or computed

Forty of the 185 tests are generated by three factories. The first crosses every operation with every operand pair and asserts the invariant rather than the answer: the calculator either returns a finite result and logs it once, or refuses and logs nothing.

```
CalculatorDynamicTest.java CalculatorNestedTest.java operations.csv

75 @TestFactory
76 @DisplayName("every operation against every operand pair")
77 Stream<DynamicNode> everyOperationAgainstEveryPair() {
78     double[][] pairs = {{6, 3}, {-6, 3}, {2.5, 0.5}, {0, 5}, {7, 0}, {1e6, 1e-6}};
79
80     return Stream.of(Operation.values()).map(operation -> dynamicContainer(
81         operation.name(),
82         Stream.of(pairs).map(pair -> dynamicTest(
83             "%s %s %s".formatted(trim(pair[0]), operation.symbol(), trim(pair[1])),
84             () -> checkOneCase(operation, pair[0], pair[1]))));
85     }
86
87 /**
88  * The invariant, not the answer: whatever the operation does, the calculator
89  * either returns a finite number and logs it once, or refuses and logs
90  * nothing. Division by zero is the only case here that must refuse.
91  */
92 private void checkOneCase(Operation operation, double left, double right) {
93     Calculator calculator = freshCalculator();
94     boolean mustRefuse = operation == Operation.DIVIDE && right == 0.0;
95
96     if (mustRefuse) {
97         assertThrows(ArithmeticException.class,
98             () -> calculator.apply(operation, left, right));
99         assertEquals(0, calculator.history().size(),
100             "a refused operation must not be logged");
101         return;
102     }
103
104     double result = calculator.apply(operation, left, right);
105     assertTrue(Double.isFinite(result), () -> "not finite: " + result);
106     assertEquals(1, calculator.history().size());
107     assertEquals(result, operation.applyTo(left, right), 1e-6,
108         "the calculator should agree with the raw operation");
109 }
110
111 /**
```

Figure 38: Four operations and six operand pairs, generating twenty-four tests from one method

dynamicContainer is what makes the failure readable. Without it a failure is reported as case 23; with it, as DIVIDE and then the pair that broke.

```
bash - @TestFactory

$ java -jar junit-platform-console-standalone.jar execute -m 'CalculatorDynamicTest#everyOperationAgainstEveryPair' ...

└─ JUnit Jupiter ✓
  └─ Calculator: dynamic tests ✓
    └─ every operation against every operand pair ✓
      └─ ADD ✓
        └─ 6 + 3 ✓
        └─ -6 + 3 ✓
        └─ 2.5 + 0.5 ✓
        └─ 0 + 5 ✓
        └─ 7 + 0 ✓
        └─ 1000000 + 1.0E-6 ✓
      └─ SUBTRACT ✓
        └─ 6 - 3 ✓
        └─ -6 - 3 ✓
        └─ 2.5 - 0.5 ✓
        └─ 0 - 5 ✓
        └─ 7 - 0 ✓
        └─ 1000000 - 1.0E-6 ✓
      └─ MULTIPLY ✓
        └─ 6 * 3 ✓
        └─ -6 * 3 ✓
        └─ 2.5 * 0.5 ✓
        └─ 0 * 5 ✓
        └─ 7 * 0 ✓
        └─ 1000000 * 1.0E-6 ✓
      └─ DIVIDE ✓
        └─ 6 / 3 ✓
        └─ -6 / 3 ✓
        └─ 2.5 / 0.5 ✓
        └─ 0 / 5 ✓
        └─ 7 / 0 ✓
        └─ 1000000 / 1.0E-6 ✓
      └─ squaring, up to the overflow boundary that is found at run time ✓
        └─ 10^0 squared is representable ✓
        └─ 10^1 squared is representable ✓
        └─ 10^10 squared is representable ✓
        └─ 10^77 squared is representable ✓
        └─ 10^154 squared is representable ✓
        └─ 10^155 squared overflows ✓
```

Figure 39: The generated tree. The names come from the values, so a failure names the case.

Dynamic tests have no lifecycle. `@BeforeEach` and `@AfterEach` run once around the whole factory, not around each generated case, so anything a case needs has to be built inside its own executable. Every factory in this class constructs its own calculator for that reason, and it is the main cost of choosing a factory over a parameterized test.

The third factory is the one that could not be written as an annotation. It searches for the largest power of ten the calculator can still square, then generates tests below that boundary that must succeed and one past it that must be refused. The boundary is not known until it has been looked for, and it comes out at ten to the 154th.

```

• CalculatorDynamicTest.java  CalculatorNestedTest.java  operations.csv

118     @TestFactory
119     @DisplayName("squaring, up to the overflow boundary that is found at run time")
120     Stream<DynamicTest> squaringUpToTheOverflowBoundary() {
121         int highestSafeExponent = 0;
122         Calculator probe = freshCalculator();
123         for (int exponent = 0; exponent <= 400; exponent++) {
124             try {
125                 double value = Math.pow(10, exponent);
126                 probe.multiply(value, value);
127                 highestSafeExponent = exponent;
128             } catch (ArithmeticException overflow) {
129                 break;
130             }
131         }
132
133         List<DynamicTest> tests = new ArrayList<>();
134         int boundary = highestSafeExponent;
135
136         // A handful of exponents below the boundary must succeed.
137         for (int exponent : new int[]{0, 1, 10, boundary / 2, boundary}) {
138             double value = Math.pow(10, exponent);
139             tests.add(dynamicTest("10^" + exponent + " squared is representable",
140                 () -> assertTrue(Double.isFinite(freshCalculator().multiply(value, value))));
141         }
142
143         // And the first one past it must be refused.
144         double past = Math.pow(10, boundary + 1);
145         tests.add(dynamicTest("10^" + (boundary + 1) + " squared overflows", () -> {
146             ArithmeticException thrown = assertThrows(ArithmeticException.class,
147                 () -> freshCalculator().multiply(past, past));
148             assertTrue(thrown.getMessage().endsWith("overflowed a double"));
149         }));
150
151         return tests.stream();
152     }

```

CalculatorDynamicTest.java Java Spaces: 4 UTF-8

Figure 40: Cases derived from a value discovered during the run

The other factory reads the same CSV file that `@CsvFileSource` reads, and parses it by hand. Adding a row to the file adds a test to both, without either class changing.

Other advanced features

Four more things the framework offers, each with a decision attached.

Feature	What it is for
@Disabled	a test that should not run, reported as skipped with a reason
@EnabledIfSystemProperty, @DisabledOnOs, @EnabledForJreRange	conditions evaluated before the run
assumeTrue, assumingThat	conditions evaluated during the run, aborting rather than failing
@Timeout, assertTimeout	a limit on the whole test, or on one block inside it
@RepeatedTest	the same test several times, to show isolation
@TestInstance(PER_CLASS)	one instance for the whole class

The distinction that matters most is between @Disabled and an assumption. @Disabled is decided before the run and the body never executes. An assumption is decided during the run, once the test can see the environment it landed in, and a test guarded by an assumption that holds is a test that really ran. Neither is the same as a failure, and reaching for @Disabled to quieten a failing test is how a suite stops being trusted.

```
113 // ----- timeouts
114
115 @Test
116 @Order(6)
117 @Timeout(value = 2, unit = TimeUnit.SECONDS)
118 @DisplayName("the whole test must finish inside two seconds")
119 void finishesQuickly() {
120     for (int i = 0; i < 50_000; i++) {
121         calculator.add(i, 1);
122     }
123     assertEquals(50_000, calculator.history().size());
124 }
125
126 @Test
127 @Order(7)
128 @DisplayName("assertTimeout measures one block rather than the whole test")
129 void assertTimeoutMeasuresABlock() {
130     // Some setup that is deliberately not part of what is being timed.
131     for (int i = 0; i < 1_000; i++) {
132         calculator.add(i, 0);
133     }
134
135     // assertTimeout runs the block on the calling thread and reports how long
136     // it took if it overruns. assertTimeoutPreemptively would run it on
137     // another thread and interrupt it, which is what you want for something
138     // that might hang – but it also means the block no longer shares the
139     // caller's thread, and anything thread-bound would break.
140     double result = assertTimeout(Duration.ofSeconds(2), () -> {
141         double total = 0;
142         for (int i = 0; i < 10_000; i++) {
143             total = calculator.add(total, 1);
144         }
145         return total;
146     });
147
148     assertEquals(10_000.0, result);
149 }
```

Figure 41: @Timeout bounds the whole test; assertTimeout bounds one block, so setup is not counted

The per-class lifecycle is the one to be careful with. It creates a single instance for the whole class, which means a field survives from one test to the next. The nested group below demonstrates that by counting up across three tests, which is precisely the shared state the default lifecycle exists to prevent.

```

• AdvancedFeaturesTest.java CalculatorTest.java LifecycleOrderTest.java

153     @Nested
154     @TestInstance(TestInstance.Lifecycle.PER_CLASS)
155     @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
156     @DisplayName("with one instance for the whole class")
157     class WithPerClassLifecycle {
158
159         /** Not static, and yet it survives between tests, because the instance does. */
160         private int callsSeen;
161
162         @Test
163         @Order(1)
164         @DisplayName("first test increments a field")
165         void first() {
166             callsSeen++;
167             assertEquals(1, callsSeen);
168         }
169
170         @Test
171         @Order(2)
172         @DisplayName("second test sees what the first one left behind")
173         void second() {
174             // Under the default PER_METHOD lifecycle this would be 1, because
175             // the instance would be new. It is 2 here, which is exactly the
176             // shared state PER_METHOD exists to prevent, and the reason this
177             // lifecycle should be reserved for expensive read-only fixtures.
178             callsSeen++;
179             assertEquals(2, callsSeen);
180         }
181
182         @Test
183         @Order(3)
184         @DisplayName("the outer @BeforeEach still runs before each of them")
185         void outerSetupStillRuns() {
186             // The fixture is rebuilt every time regardless of the lifecycle, so
187             // the history is empty even though callsSeen is not.
188             assertEquals(0, calculator.history().size());
189             callsSeen++;
190             assertEquals(3, callsSeen);
191         }
192     }
193 }

```

AdvancedFeaturesTest.java Java

Spaces: 4 UTF-8

Figure 42: Shared state, on purpose. Under the default lifecycle each of these would see 1.

Worth noting in that group: the outer `@BeforeEach` still runs before every test regardless of the lifecycle, so the calculator is rebuilt each time even though the counter is not. The lifecycle changes how often the class is instantiated, not how often the callbacks fire.

```
bash - skips, conditions and the per-class lifecycle

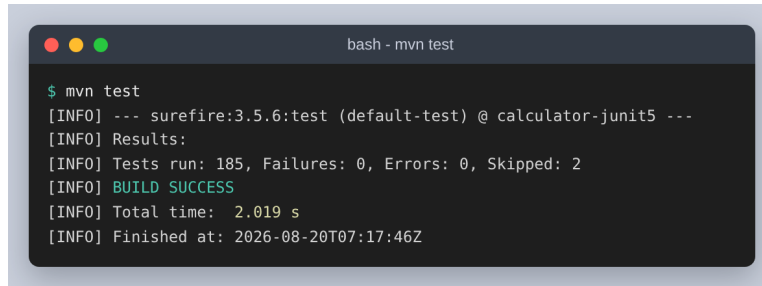
$ java -jar junit-platform-console-standalone.jar execute -c AdvancedFeaturesTest
└─ JUnit Jupiter ✓
  └─ Advanced JUnit 5 features ✓
    └─ square root, once the calculator has one ~ Square root is not part of the assignment; kept to show a skip is reported, not hidden
    └─ a test that only makes sense with a system property set ~ System property [run.slow.tests] does not exist
    └─ conditions can also key off the OS and the JRE ✓
    └─ an assumption aborts the test rather than failing it ✓
    └─ assumingThat guards one part of a test without skipping the rest ✓
    └─ the whole test must finish inside two seconds ✓
    └─ assertTimeout measures one block rather than the whole test ✓
    └─ with one instance for the whole class ✓
      └─ first test increments a field ✓
      └─ second test sees what the first one left behind ✓
      └─ the outer @BeforeEach still runs before each of them ✓

Test run finished after 209 ms
[ 3 containers found      ]
[ 0 containers skipped    ]
[ 3 containers started    ]
[ 0 containers aborted    ]
[ 3 containers successful ]
[ 0 containers failed     ]
[ 10 tests found         ]
[ 2 tests skipped        ]
[ 8 tests started        ]
[ 0 tests aborted        ]
[ 8 tests successful     ]
[ 0 tests failed         ]
```

Figure 43: Two skips, each with its reason, and the three tests sharing an instance

Results

185 tests, no failures, two skipped. The two skips are the `@Disabled` test and the one behind a system property, and both are reported with the reason they did not run.

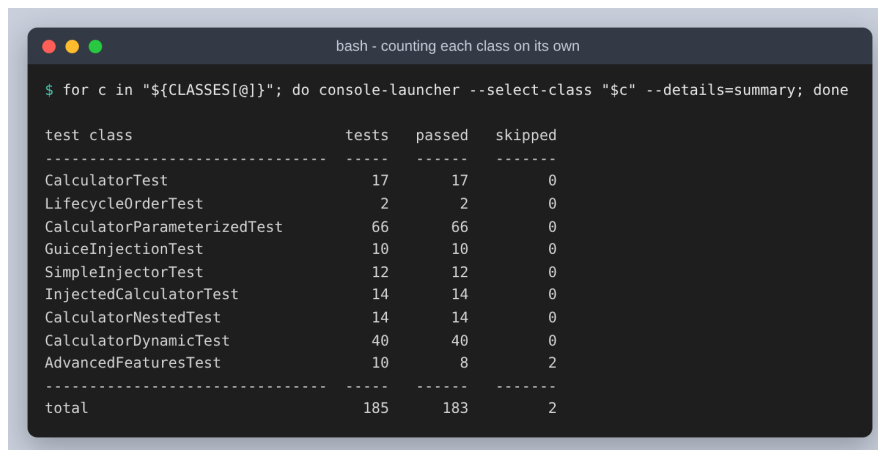
A terminal window titled "bash - mvn test" showing the output of the command `$ mvn test`. The output includes information about the Surefire version, test results (185 tests run, 0 failures, 0 errors, 2 skipped), a success message, total time (2.019 s), and the finish time (2026-08-20T07:17:46Z).

```
bash - mvn test

$ mvn test
[INFO] --- surefire:3.5.6:test (default-test) @ calculator-junit5 ---
[INFO] Results:
[INFO] Tests run: 185, Failures: 0, Errors: 0, Skipped: 2
[INFO] BUILD SUCCESS
[INFO] Total time: 2.019 s
[INFO] Finished at: 2026-08-20T07:17:46Z
```

Figure 44: The full run

Surefire reports each `@Nested` class as its own test set, and does not reliably attribute an enclosing class's own tests to the enclosing line, so its per-class breakdown shows zero for classes that do have tests. The totals are right and the breakdown is not, so the table below counts each class on its own through the console launcher instead.

A terminal window titled "bash - counting each class on its own" showing a script that iterates over classes and uses the console launcher to count tests. The output is a table with columns: test class, tests, passed, and skipped. The total is 185 tests, 183 passed, and 2 skipped.

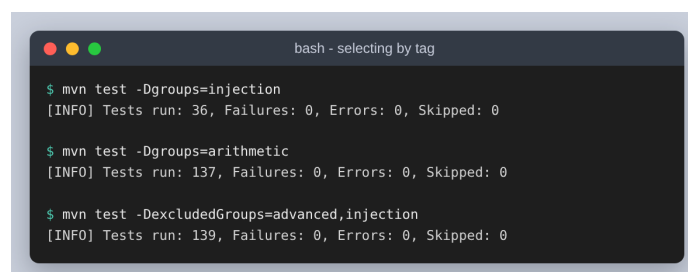
```
bash - counting each class on its own

$ for c in "${CLASSES[@]}"; do console-launcher --select-class "$c" --details=summary; done

test class          tests  passed  skipped
-----
CalculatorTest       17      17       0
LifecycleOrderTest   2        2       0
CalculatorParameterizedTest 66      66       0
GuiceInjectionTest   10      10       0
SimpleInjectorTest   12      12       0
InjectedCalculatorTest 14      14       0
CalculatorNestedTest  14      14       0
CalculatorDynamicTest 40      40       0
AdvancedFeaturesTest 10       8        2
-----
total                185    183      2
```

Figure 45: Where the 185 tests come from

The classes are tagged, which makes it possible to run part of the suite without naming classes. The three counts are consistent: 137 arithmetic plus 36 injection plus 10 advanced is 183, and the remaining two are `LifecycleOrderTest`, which carries no tag.

A terminal window titled "bash - selecting by tag" showing three commands to run tests by tag. The first command runs injection tests (36), the second runs arithmetic tests (137), and the third runs all tests except advanced and injection (139).

```
bash - selecting by tag

$ mvn test -Dgroups=injection
[INFO] Tests run: 36, Failures: 0, Errors: 0, Skipped: 0

$ mvn test -Dgroups=arithmetic
[INFO] Tests run: 137, Failures: 0, Errors: 0, Skipped: 0

$ mvn test -DexcludedGroups=advanced,injection
[INFO] Tests run: 139, Failures: 0, Errors: 0, Skipped: 0
```

Figure 46: Selecting by tag

Design decisions, collected

- Four operations funnel into one method, so the validation cannot be bypassed by choosing a different entry point.
- Division by zero is checked explicitly, because IEEE-754 makes it infinity rather than an error and the JVM will not object.
- A result that overflows is an error rather than an Infinity handed back to the caller, and a NaN operand is rejected before any arithmetic so the message says where the problem entered.
- Rounding is an injected policy, which is what lets the tests show both the tidy result and the raw one.
- The operation log is injected and stateful, which is what makes `@AfterEach` necessary rather than decorative.
- The enum is the single source of truth for the operation set, so the parameterized and dynamic tests cannot fall behind it.
- The suite lists its classes explicitly, so the class that fails on purpose cannot be picked up by accident.
- The suite and the failing demonstration are excluded from the default run and given profiles, so the reported total is not double counted.
- The extension caches per test rather than per class, after the class-level version was found to share one calculator with every test in the class.

Running it

```
cd calculator-junit5

mvn test                # 185 tests
mvn test -Psuite        # the same, through @Suite
mvn test -Pshow-failure # the deliberate failures
mvn test -Dgroups=Injection # by tag
mvn test -Dtest=LifecycleOrderTest
mvn compile exec:java    # the application

./scripts/capture.sh      # regenerate results/
python3 scripts/make_figures.py # regenerate figures/
python3 build.py          # regenerate this document
```

Java 21 is used rather than the newer release available, because Guice 7 generates classes at run time and is only tested against LTS releases.